

Categorizing Cyber-Attacks: Analysis of Industry, Event-Type, Economic and Criminological Factors

SR - Behavioral and Social Sciences

Aaron D'Souza

Table of Contents

Contents

Abstract	2
Acknowledgements	3
Introduction	4
Review of Literature.....	6
Statement of Problem	10
Hypothesis.....	11
Materials.....	12
Procedure.....	13
Results and Statistical Analysis.....	14
Discussion	36
Conclusion.....	38
Recommendations	40
Bibliography.....	41
Appendix 1: Google Colab python Code.....	43

Abstract

This work determines which industries are most affected by cyber-attacks, which types of attacks are most prevalent, and underlying causes.

The hypothesis is that the healthcare industry experiences the greatest number of cyber-attacks due to a large concentration of personal data, recurrent access to the data by numerous individuals, and potentially outdated systems.

First, data was obtained from the publicly available Center for International and Security Studies at Maryland (CISSM) website. Then, data analysis and visualization were performed in Python programming using the Google-Colab environment to identify insights and classify cyber-attacks based on economic and criminological factors.

The analysis indicates that in recent years the healthcare and social assistance industry is victim to the greatest number of cyber-attacks due to increasing adoption of Electronic Health Records. However, across the decade the public administration industry had the highest number of cyber-attacks. This finding is close but not exactly consistent with the research hypothesis. The work then proceeded with a detailed analysis of cyber-attacks and their classification by attack type, motive and actors, and to explain the discrepancy versus the hypothesis. The conclusion of the detailed analysis is that: (1) Considering exploitative attacks or those with financial motive, healthcare has the greatest number of cyber-attacks, whereas (2) Politically motivated disruptive attacks resulted in public administration having the greatest number of cyber-attacks.

Such classification of attacks is expected to assist industries and governments safeguard against cyber-attacks. Further, such a study illustrates the efficacy of data science and data visualization in understanding social science.

Acknowledgements

I would like to thank my teachers at The Cambridge School, especially my science teacher, Mrs. Mayne, for their help and support. I would like to thank my computer science mentor, Mr. Abulfaz. I would also like to thank my parents for their love and support.

Introduction

Research problem:

My research problem is to determine which industries are most affected by cyber-attacks, which types of attacks are most prevalent, and understanding causes for certain industries being more vulnerable and attacked. Then, compare with other industries with lower cyber events to find unique causes.

I became interested in this area from my participation in cybersecurity competitions, and classes on data science and artificial intelligence. Upon reading research papers, I became interested in pursuing this topic since it combines my interests in cybersecurity and data science.

This topic is very important since cyber-attacks affect real people threatening their privacy, financial security, and every aspect of one's life tied to data and/or technology. Additionally, cyber-attacks on government agencies can threaten national security affecting public safety, social infrastructure, and defense.

Although important, as far as I know, this topic has not been investigated in a prior GSDSEF

Prior work:

I read several papers listed in the reference section of this document. One relevant reference is "A study of cyber-attacks: In the healthcare sector" by Karuna Bhosale. This reference highlights challenges including endpoint device management, security awareness issues, and inadequate protection of connected medical devices and hospital information systems, demonstrating a need for enhanced cybersecurity strategies and solutions.

The limitation of prior work is that it does not identify which industry sector is most vulnerable to cyber-attacks and why.

My research addresses these limitations through a detailed data science analysis of cyber-attack records to identify which industries are most impacted and the underlying causes

Hypothesis

My hypothesis is that healthcare would be the victim with the greatest number of cyber-attacks with the primary type being exploitative due to a large concentration of personal data, recurrent access to the data by numerous individuals, and potentially outdated systems.

The basis of hypothesis is my experience and knowledge of cyber-attacks from participating in Cyberpatriot, Southern California Cyber Cup and National Cyber League competitions, my personal visits to doctors' offices and reading current news events on cyber-attacks. As I researched this topic further, I read literature which supported my hypothesis and helped me refine it, e.g., U.S. Center for Disease Control article which explains that in the U.S. a vast majority of physicians use electronic medical records; thus, allowing more cyber attacks

Review of Literature

I read several papers listed in the reference section of this document.

[1] One relevant reference is “A study of cyber-attacks: In the healthcare sector” published in IEEE Xplore by Karuna Bhosale. This reference highlights challenges including endpoint device management, security awareness issues, and inadequate protection of connected medical devices and hospital information systems, demonstrating a need for enhanced cybersecurity strategies and solutions. Importantly, they identify that healthcare sectors are “extensively sensible and realistic to cyber-attacks, wherein disorganization in operations or even divulgence of patient's personal information can have profound consequences” (Bhosale, Nenova and Iliev, 2021). Overall, Bhosale’s study indicates the specific healthcare cybersecurity greater vulnerability.

[2] More specifically, “Cybersecurity in healthcare: A narrative review of trends, threats and ways forward” published in Maturitas by Lynne Coventry identifies two primary reasons for healthcare being an attractive target for cyber attacks more possible now due to increased connectivity of medical technology. The two reasons are it being a “rich source of valuable data” and its defenses being weak. Procedurally, Coventry explains that breaches involve stealing health information and ransomware attack while legislation to secure systems “has historically been lax” (Coventry and Branley, 2018).

[3] In fact, the Electronic Privacy Information Center identifies several specific problems in the current Healthcare system in respect to privacy. First, “the bulk of medical records are covered by various, inconsistent, and often ineffectual state laws” meaning that laws for enforcement and punishment are insufficient to deterrence. Also, EPIC identifies that numerous parties accessing patient databases and large amounts of data in small space leads to greater vulnerability to cyber-attack (Electronic Privacy Information Center, 2024).

[4] The Centers for Disease Control and Prevention (CDC) documented a significant increase in telemedicine adoption among office-based physicians, rising from 15.4% in 2019 to 85.9% in 2021. This surge

was seen across many specialties, with 91.4% of primary care physicians, 87.2% of medical specialty physicians, and 74.8% of surgical specializing physicians were utilizing telemedicine technology in 2021. This mass adoption underlines the healthcare sector's rapid integration of electronic tools, in effort to enhance patient care accessibility. However, also introducing new cybersecurity challenges that must be addressed to protect sensitive health information.

[5] According to a new report by Sophos, "The State of Ransomware in Healthcare 2024," the rate of ransomware attacks against healthcare organizations has hit a four-year high "since 2021," with 67% hit in the past year only compared to 60% in 2023. Especially, the report shows less than a quarter (22%) of ransomware victims' complete recovery in a week or less which is a decline from 47% in 2023 this means attacks are becoming more severe this year as they are taking longer to recover from. Specifically, Sophos identifies the average cost on healthcare organizations to recover from such an attack has grown to over \$2.5 million in 2024 from \$2.2 million in 2023 (Sophos, 2024).

[6] Moving on to public administration, cyber-attacks on public administration sectors, according to the CyberPeace Institute, "threaten e-government activities" and endanger "the protection of sensitive government and personal data." Such attacks also can hold up key public services with serious social and economic consequences (CyberPeace Institute, n.d.).

[7] For example, over 30 government institutions in Costa Rica, including the Ministry of Finance and the Social Security Fund, were seized in the 2022 Costa Rican ransomware attack, which stopped tax collection and disrupted health services. The attack crippled Costa Rica's digital infrastructure for months leading to President Chaves stating: "We are at war, and that is not an exaggeration" (Murray and Srivastava, 2022). This episode shows us the vital importance of strong cybersecurity measures within public administration to safeguard national functions and citizen welfare.

[8] Specifically on Russian attacks, the Cybersecurity and Infrastructure Security Agency published Advisory AA24-249A. It outlines cyber operations performed by Russia's GRU Unit 29155 as far back as 2020. These operations have been directed against critical infrastructure systems across the world for spying and sabotage which included using WhisperGate malware against Ukrainian entities in January 2022. CISA

suggests that entities “prioritize routine system updates and remediate known exploited vulnerabilities” to reduce such risks (CISA, 2024).

[9] Moreover, a June 2024 Cyber Threat Intelligence Integration Center (CTIIC) report highlights that between November 2023 and April 2024, Iran-linked and pro-Russia cyber actors "gained access to and in some cases have manipulated critical US industrial control systems (ICS) in the food and agriculture, healthcare, and water and wastewater sectors." The report emphasizes that such incidents "highlight a potential public safety threat and an avenue for malicious cyber actors to cause physical damage and deny critical services." Circumstances like "outdated software, poor password security, the use of default credentials, and limited resources for system updates" render ICS devices extremely vulnerable. The CTIIC calls for enhancing the cybersecurity defense of these essential infrastructures.

[10] More on Russia, cyber attacks have become part of their military strategy. In Chapter 5 of *Understanding Cyber Conflict: Fourteen Analogies*, "Cyber War and Information War à la Russe," author Stephen Blank examines Russia's integration of cyber war and information war into a unified strategy for power projection. Referring to Russian activity in Estonia, Georgia, and Ukraine between 2006 and 2016, Blank argues that Russia does not distinguish between IW and cyber warfare; instead, it treats them as part of "information confrontation" (*Informatsionoye protivoborstvo*). It is a strategy that leverages synchronized cyber-attacks, propaganda, and psychological operations to achieve political and military objectives. For instance, in the 2015 cyber assault on Ukraine's power grid, Russian perpetrators leveraged malware to disrupt electricity transmission, showing how perfectly these types of cyber and physical operations coexist with one another in Russian strategy.

[10] Moving on to the causal side, “Economics of Information Security” by Anderson and Moore Anderson and Moore argues that “incentives are becoming as important to dependability as technical design.” They highlight that misaligned incentives among stakeholders leading to underinvestment in cybersecurity defense systems: "legal theorists have long known that liability should be assigned to the party that can best manage the risk." This economic perspective points to systemic issues that may contribute to the healthcare sector's susceptibility to cyber threats (Anderson and Moore, 2006).

The limitation of prior work is that it does not identify which industry sector is most vulnerable to cyber-attacks and why.

My research addresses these limitations through a detailed data science analysis of cyber-attack records to identify which industries are most impacted and the underlying causes

Statement of Problem

My research problem is to determine which industries are most affected by cyber-attacks, which types of attacks are most prevalent, and understanding causes for certain industries being more vulnerable and attacked. Then, compare with other industries with lower cyber events to find unique causes.

Hypothesis

My hypothesis is that the healthcare industry would be the victim with the greatest number of cyber-attacks with the primary type of cyber-attacks on healthcare being exploitative due to a large concentration of personal data, recurrent access to data by numerous individuals, and potentially outdated systems.

I based my hypothesis on my experience and knowledge of cyber-attacks from participating in Cyberpatriot, Southern California Cyber Cup and National Cyber League competitions, my personal visits to doctors' offices and reading current news events on cyber-attacks.

As I researched this topic further, I read literature which supported my hypothesis and helped me refine it, e.g., U.S. Center for Disease Control article which explains that in the U.S. a vast majority of physicians use electronic medical records; thus, allowing more cyber attacks

Materials

This data science project uses data sourced from Center for International and Security Studies at University of Maryland(CISSM).

The data, “Cyber Events Database” is a record of scraped publications of cyber-attack events occurring since 2014 to 2024 with over 14,000 recorded events. I downloaded the cyber events dataset that I used from this link: <https://cissm.umd.edu/cyber-events-database>

I also used data from The World Bank’s Poverty & Inequality Indicators (PIP) which include various economic describing-variables per country including GDP, wealth gap, gini coefficient, etc. I downloaded the PIP dataset that I used from this link: <https://pip.worldbank.org/poverty-calculator>

The cyber events are classified by date, actor type (criminal, hacktivist, nation-state, undetermined), actor, organization (which organization was impacted by the event), industry, motive (protest, sabotage, espionage, financial, undetermined), event type (disruptive, exploitative, mixed undetermined), event description, source url, target country, and actor country. The data collects only publicly available information, using python to “scrape” data. Then researchers from CISSM reviewed and categorized the data to ensure the source material information is reflected consistently in the data and the event identified fits their definition of a cyber event. The industries were categorized per the North American Industry Classification System.

I did the data science and data visualization analysis for my project using the Python program in the Google colab environment which I accessed online from my MacBook. The code for my project is available at the link:

https://colab.research.google.com/drive/1ThXrjvK0lxD_CQ6YjgvV7VEjMUwJWke7?usp=sharing

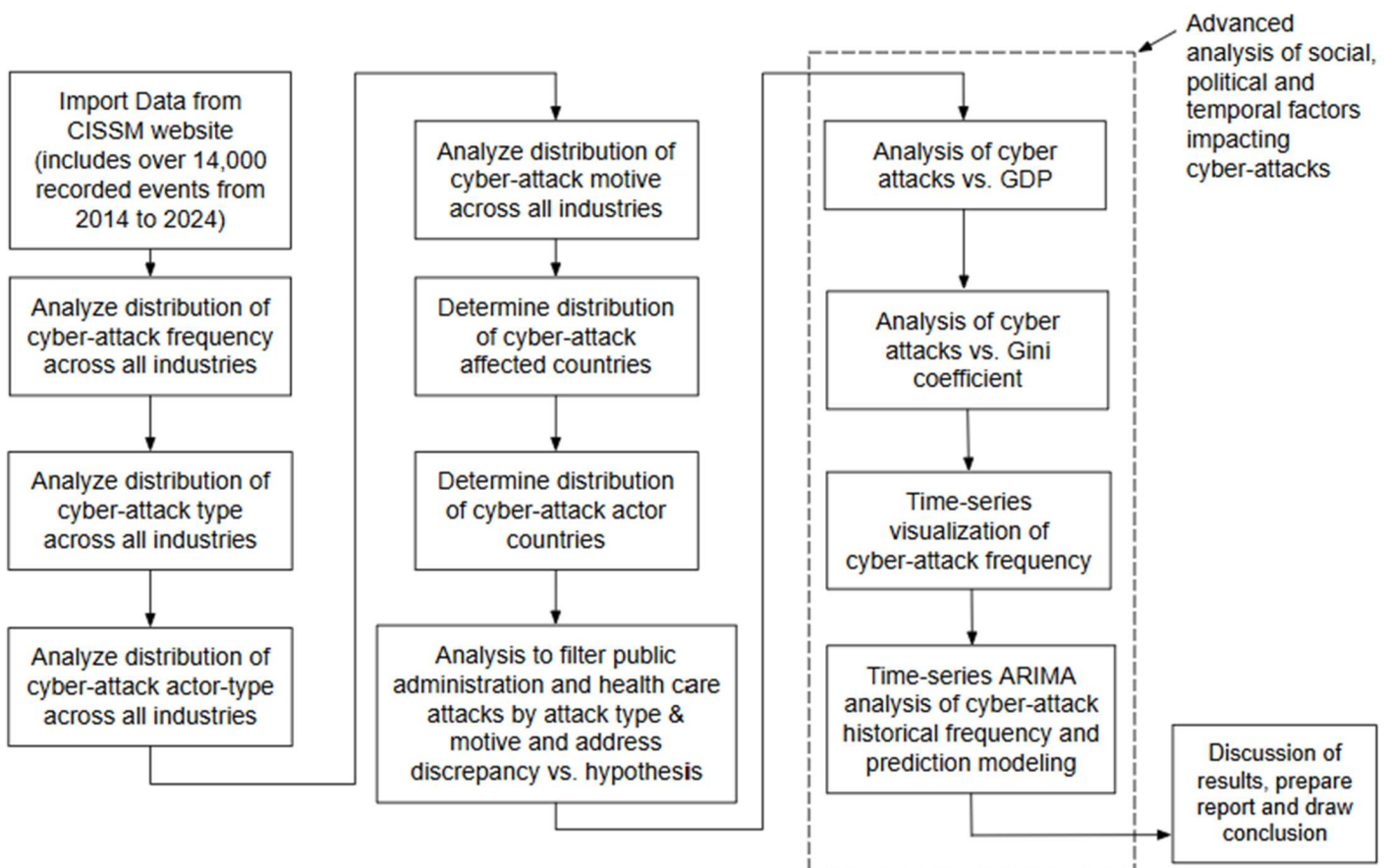
I also used Google Docs, Google Slides, Google Sheets to compile my data and write the reports for my project.

Procedure

I Investigated the research problem using the following procedural steps shown with a block diagram flow chart below.

Implementation of these steps in my Python program in the Google colab environment is at the link:

https://colab.research.google.com/drive/1ThXrjvK0lxD_CQ6YjgvV7VEjMUwJWke7?usp=sharing



Results and Statistical Analysis

This data science project uses data sourced from Center for International and Security Studies at University of Maryland (CISSM). The results of the data analysis and data visualization are on the following slides.

A few relevant definitions:

Exploitative cyber-attack: An attack wherein attacks exploit vulnerabilities in software, hardware, or network systems. These exploits enable unauthorized access, often bypassing security controls, and leading to compromised data or operations.

Disruptive cyber-attack: Attack which does not aim to steal data or breach security systems but to render the target inoperable.

Result 1: Cyber-Attacks from 2014 to 2023 by industry

Table 1: Number of Cyber-Attacks from 2014 to 2023 by industry

Industry Number	Industry Name	Number of Cyber-Attacks from 2014 to 2023
1	Public Administration	2648
2	Health Care and Social Assistance	1865
3	Information	1441
4	Finance and Insurance	1340
5	Educational Services	1322
6	Professional, Scientific, and Technical Services	1170
7	Other Services (except Public Administration)	937
8	Manufacturing	645
9	Retail Trade	469
10	Transportation and Warehousing	443
11	Arts, Entertainment, and Recreation	420
12	Accommodation and Food Services	306
13	Utilities	270
14	Undetermined	209
15	Administrative and Support and Waste Management and Remediation Services	179
16	Wholesale Trade	107
17	Real Estate and Rental and Leasing	84
18	Mining, Quarrying, and Oil and Gas Extraction	73
19	Construction	47
20	Health Care and Social Services	27
21	Management of Companies and Enterprises	20
22	Agriculture, Forestry, Fishing and Hunting	18
23	Medusa	1

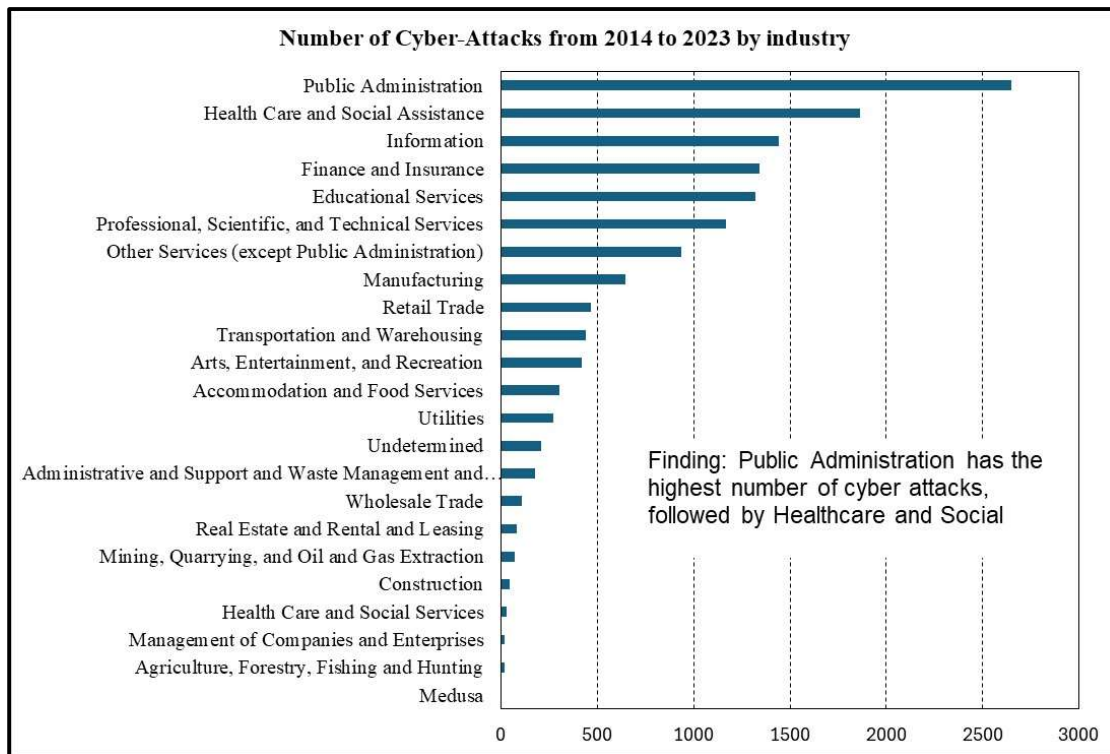


Figure 1: Number of Cyber-Attacks from 2014 to 2023 by industry

Result 2: Cyber-Attacks from 2014 to 2023 by attack type

Table 2: Number of Cyber-Attacks from 2014 to 2023 by attack type

Attack Type Number	Attack Type Name	Number of Cyber-Attacks from 2014 to 2023	Percentage
1	Disruptive	4339	30.95%
2	Exploitive	7000	49.93%
3	Mixed	2500	17.83%
4	Undetermined	181	1.29%

Total = 14020

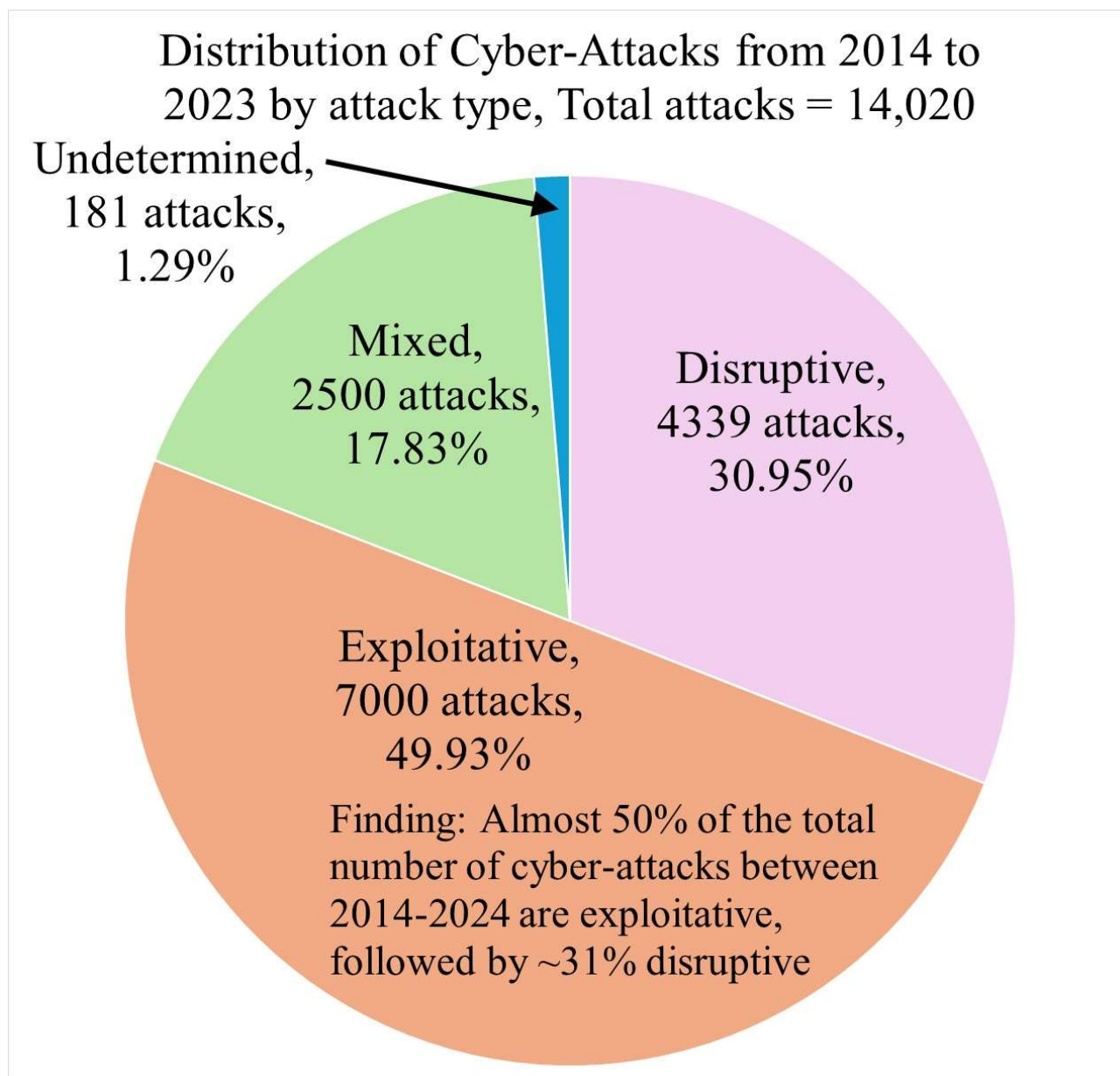


Figure 2: Number of Cyber-Attacks from 2014 to 2023 by attack type

Result 3: Cyber-Attacks from 2014 to 2023 by industry and attack-type

Table 3: Number of Cyber-Attacks from 2014 to 2023 by industry and attack-type

Industry Number	Industry Name	Disruptive	Exploitative	Mixed	Undetermined
1	Public Administration	1249	1041	310	44
2	Health Care and Social Assistance	334	1070	436	25
3	Information	577	709	138	9
4	Finance and Insurance	300	855	173	12
5	Educational Services	438	583	276	25
6	Professional, Scientific, and Technical Services	188	649	324	9
7	Other Services (except Public Administration)	291	555	86	5
8	Manufacturing	167	204	264	9
9	Retail Trade	57	321	84	7
10	Transportation and Warehousing	226	136	70	8
11	Arts, Entertainment, and Recreation	163	213	40	4
12	Accommodation and Food Services	38	221	43	4
13	Utilities	118	81	60	10
14	Undetermined	56	112	39	2
15	Administrative and Support and Waste Management and Remediation Services	45	96	35	3
16	Wholesale Trade	24	52	29	2
17	Real Estate and Rental and Leasing	16	40	28	0
18	Mining, Quarrying, and Oil and Gas Extraction	32	30	10	1
19	Construction	9	10	27	1
20	Health Care and Social Services	0	11	11	1
21	Management of Companies and Enterprises	5	8	7	0
22	Agriculture, Forestry, Fishing and Hunting	6	3	9	0
23	Medusa	0	0	1	0

Result 3 Continued: Cyber-Attacks from 2014 to 2023 by industry and attack-type

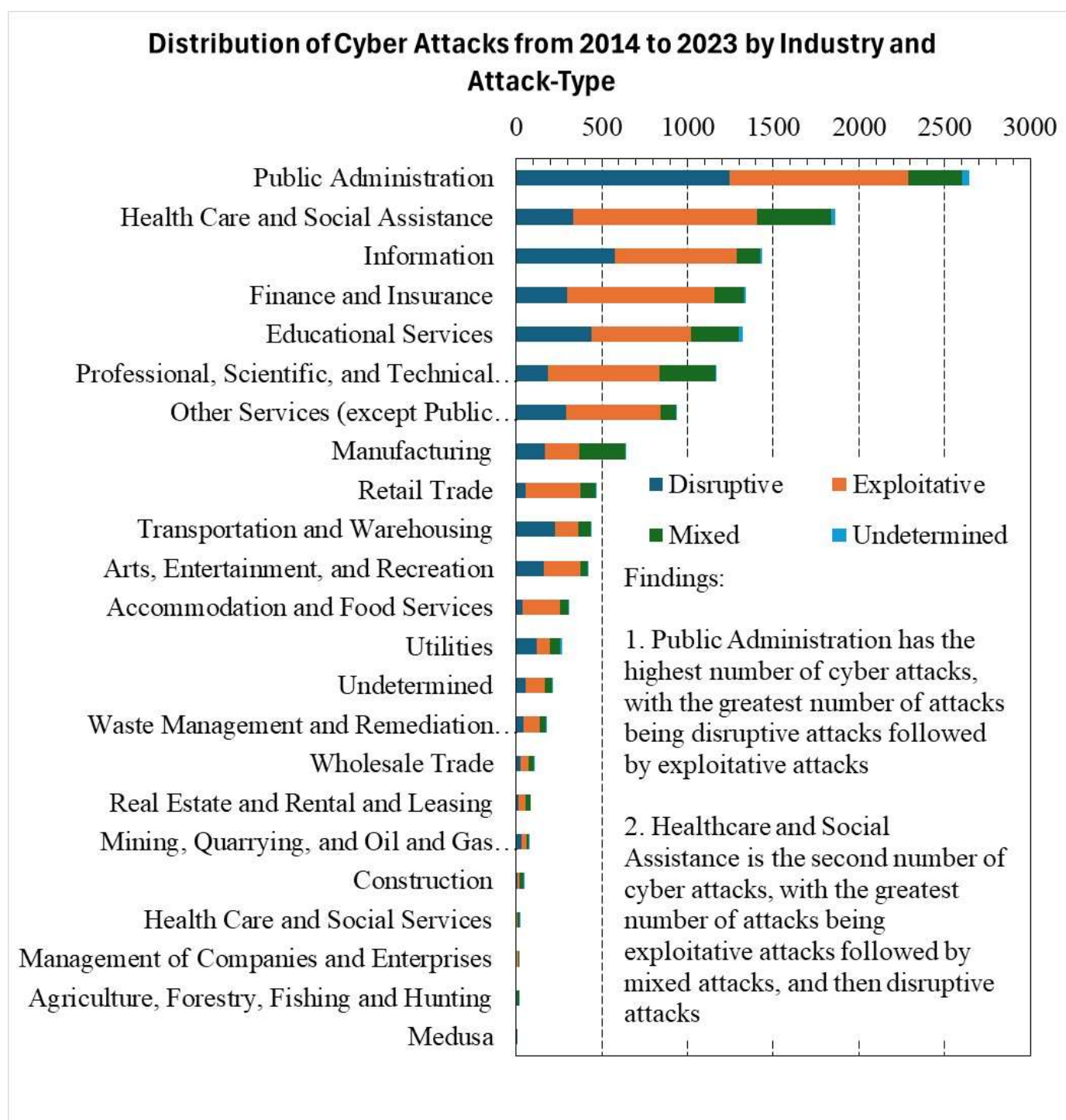


Figure 3: Number of Cyber-Attacks from 2014 to 2023 by industry and attack-type

Result 4: Cyber-Attacks from 2014 to 2023: Percentage of Attack-Type for each Industry

Table 4: Cyber-Attacks from 2014 to 2023: Percentage of Attack-Type for each Industry

Industry Number	Industry Name	Disruptive	Exploitative	Mixed	Undetermined	Total Industry Cyber-Attacks 2014 to 2023
1	Public Administration	1249	1041	310	44	2644
2	Health Care and Social Assistance	334	1070	436	25	1865
3	Information	577	709	138	9	1433
4	Finance and Insurance	300	855	173	12	1340
5	Educational Services	438	583	276	25	1322
6	Professional, Scientific, and Technical Services	188	649	324	9	1170
7	Other Services (except Public Administration)	291	555	86	5	937
8	Manufacturing	167	204	264	9	644
9	Retail Trade	57	321	84	7	469
10	Transportation and Warehousing	226	136	70	8	440
11	Arts, Entertainment, and Recreation	163	213	40	4	420
12	Accommodation and Food Services	38	221	43	4	306
13	Utilities	118	81	60	10	269
14	Undetermined	56	112	39	2	209
15	Waste Management and Remediation Services	45	96	35	3	179
16	Wholesale Trade	24	52	29	2	107
17	Real Estate and Rental and Leasing	16	40	28	0	84
18	Mining, Quarrying, and Oil and Gas Extraction	32	30	10	1	73
19	Construction	9	10	27	1	47
20	Health Care and Social Services	0	11	11	1	23
21	Management of Companies and Enterprises	5	8	7	0	20
22	Agriculture, Forestry, Fishing and Hunting	6	3	9	0	18
23	Medusa	0	0	1	0	1

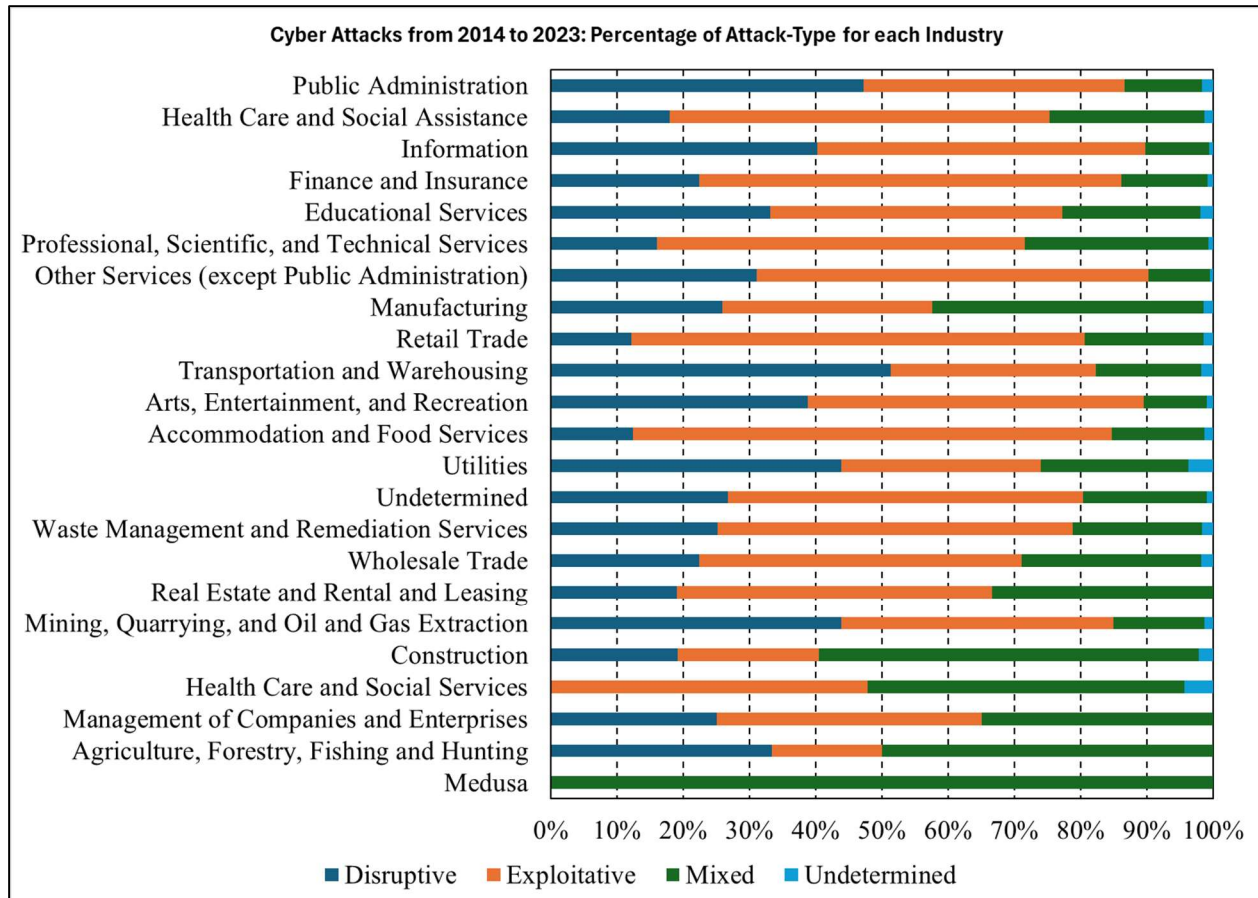


Figure 4: Cyber-Attacks from 2014 to 2023: Percentage of Attack-Type for each Industry

Result 5: Cyber-Attacks from 2014 to 2023 by attack Motive

Table 5: Number of Cyber-Attacks from 2014 to 2023 by attack motive

Attack Motive Number	Attack Motive Name	Number of Cyber-Attacks from 2014 to 2023	Percentage
1	Financial	8034	57.22%
2	Undetermined	3119	22.21%
3	Protest	1680	11.96%
4	Political-Espionage	679	4.84%
5	Sabotage	340	2.42%
6	Industrial-Espionage	94	0.67%
7	Personal Attack	93	0.66%
8	Protest,Financial	1	0.01%
9	Protest;Political-Espionage	1	0.01%
Total =		14041	

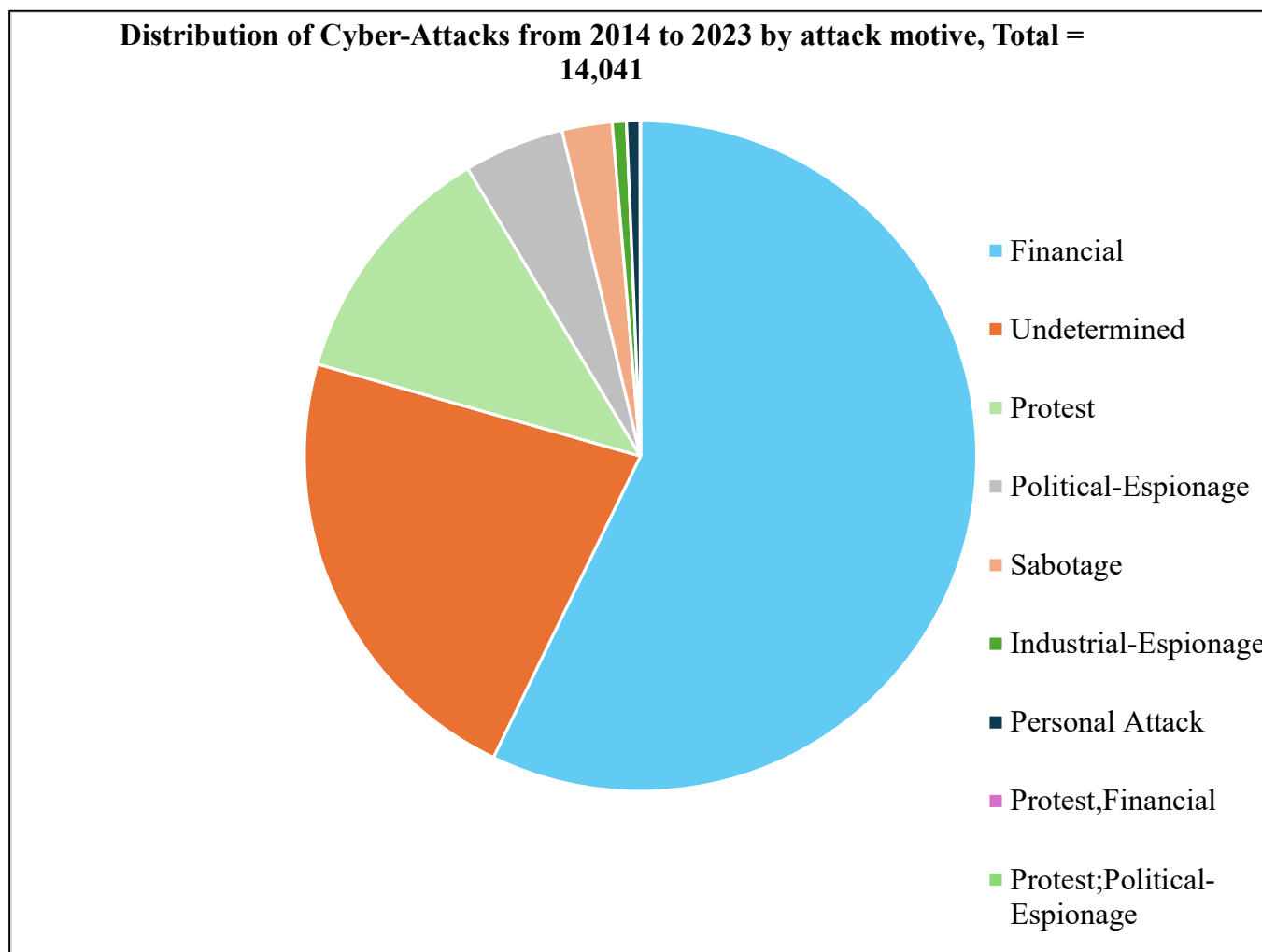


Figure 5: Number of Cyber-Attacks from 2014 to 2023 by attack motive

Result 6: Cyber-Attacks from 2014 to 2023 by industry and motive

Table 6: Number of Cyber-Attacks from 2014 to 2023 by industry and motive

Industry Number	Industry Name	Financial	Protest	Political Espionage	Undetermined Motive	Sabotage	Industrial Espionage	Personal Attack	Total Industry Cyber-Attacks 2014 to 2023
1	Public Administration	871	688	313	682	75	14	4	2647
2	Health Care and Social Assistance	1527	33	5	289	7	3	1	1865
3	Information	545	186	79	537	82	6	6	1441
4	Finance and Insurance	971	165	17	151	35	0	1	1340
5	Educational Services	888	57	17	350	2	4	3	1321
6	Professional, Scientific, and Technical Services	807	44	27	259	11	20	2	1170
7	Other Services (except Public Administration)	269	108	147	319	24	0	70	937
8	Manufacturing	469	79	5	59	5	28	0	645
9	Retail Trade	382	8	4	68	6	0	1	469
10	Transportation and Warehousing	202	127	7	78	27	2	0	443
11	Arts, Entertainment, and Recreation	225	23	3	157	6	1	5	420
12	Accommodation and Food Services	263	9	3	30	1	0	0	306
13	Utilities	122	50	11	43	41	3	0	270
14	Undetermined	114	16	36	28	7	8	0	209
15	Administrative and Support and Waste Management and Remediation Services	126	27	2	21	0	3	0	179
16	Wholesale Trade	78	15	0	13	0	1	0	107
17	Real Estate and Rental and Leasing	63	5	0	15	1	0	0	84
18	Mining, Quarrying, and Oil and Gas Extraction	27	26	2	9	8	1	0	73
19	Construction	34	8	0	5	0	0	0	47
20	Health Care and Social Services	27	0	0	0	0	0	0	27
21	Management of Companies and Enterprises	13	2	1	3	1	0	0	20
22	Agriculture, Forestry, Fishing and Hunting	10	4	0	3	1	0	0	18

Result 6 Continued: Cyber-Attacks from 2014 to 2023 by industry and motive

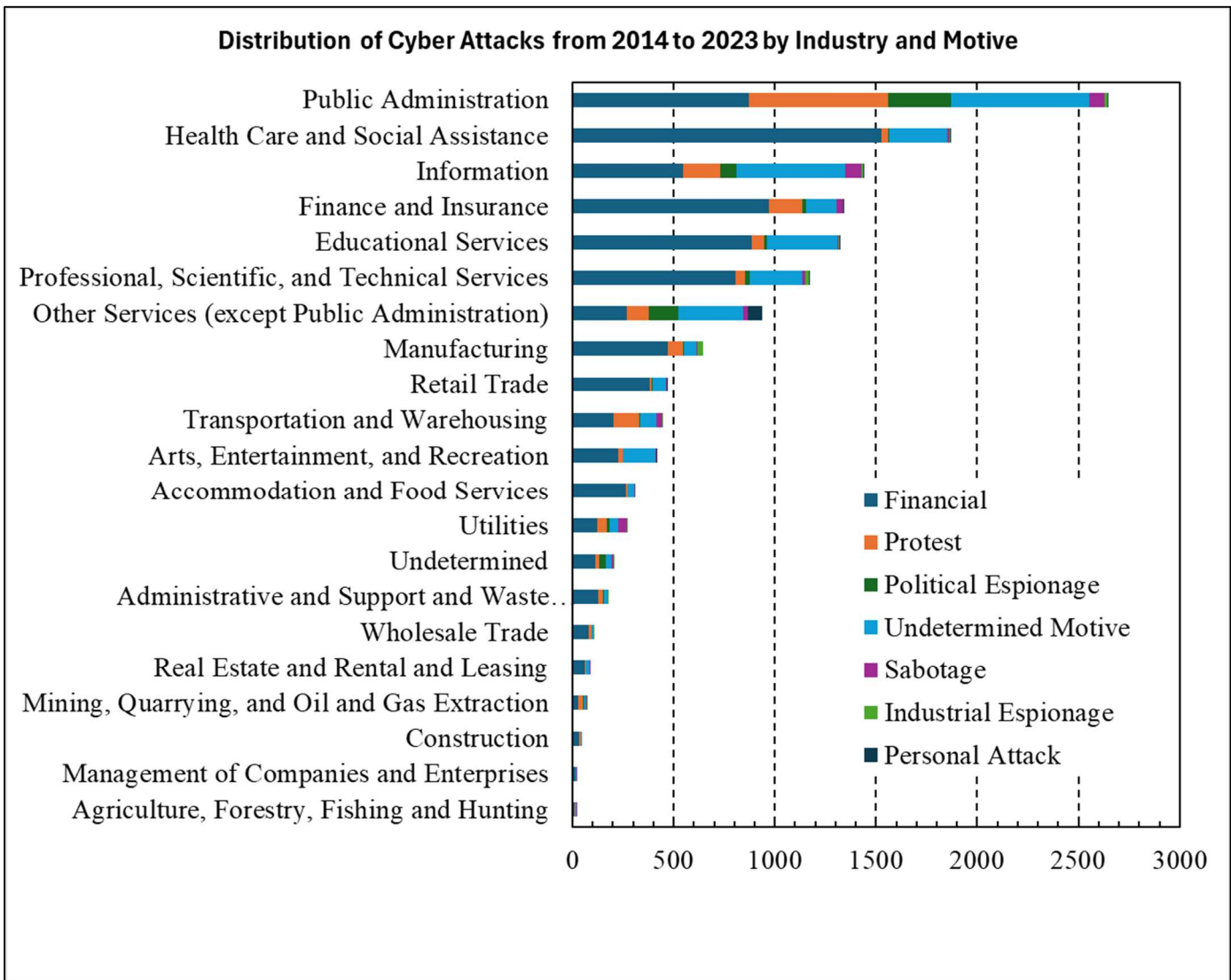


Figure 6: Number of Cyber-Attacks from 2014 to 2023 by industry and Motive

Result 7: Cyber-Attacks from 2014 to 2023: Percentage of Attack-Motive for each Industry

Table 7: Cyber-Attacks from 2014 to 2023: Percentage of Attack-Motive for each Industry

Industry Number	Industry Name	Percentage of Attacks by Motive						
		Financial	Protest	Political Espionage	Undetermined Motive	Sabotage	Industrial Espionage	Personal Attack
1	Public Administration	32.91%	25.99%	11.82%	25.77%	2.83%	0.53%	0.15%
2	Health Care and Social Assistance	81.88%	1.77%	0.27%	15.50%	0.38%	0.16%	0.05%
3	Information	37.82%	12.91%	5.48%	37.27%	5.69%	0.42%	0.42%
4	Finance and Insurance	72.46%	12.31%	1.27%	11.27%	2.61%	0.00%	0.07%
5	Educational Services	67.22%	4.31%	1.29%	26.50%	0.15%	0.30%	0.23%
6	Professional, Scientific, and Technical Services	68.97%	3.76%	2.31%	22.14%	0.94%	1.71%	0.17%
7	Other Services (except Public Administration)	28.71%	11.53%	15.69%	34.04%	2.56%	0.00%	7.47%
8	Manufacturing	72.71%	12.25%	0.78%	9.15%	0.78%	4.34%	0.00%
9	Retail Trade	81.45%	1.71%	0.85%	14.50%	1.28%	0.00%	0.21%
10	Transportation and Warehousing	45.60%	28.67%	1.58%	17.61%	6.09%	0.45%	0.00%
11	Arts, Entertainment, and Recreation	53.57%	5.48%	0.71%	37.38%	1.43%	0.24%	1.19%
12	Accommodation and Food Services	85.95%	2.94%	0.98%	9.80%	0.33%	0.00%	0.00%
13	Utilities	45.19%	18.52%	4.07%	15.93%	15.19%	1.11%	0.00%
14	Undetermined	54.55%	7.66%	17.22%	13.40%	3.35%	3.83%	0.00%
15	Administrative and Support and Waste Management and Remediation Services	70.39%	15.08%	1.12%	11.73%	0.00%	1.68%	0.00%
16	Wholesale Trade	72.90%	14.02%	0.00%	12.15%	0.00%	0.93%	0.00%
17	Real Estate and Rental and Leasing	75.00%	5.95%	0.00%	17.86%	1.19%	0.00%	0.00%
18	Mining, Quarrying, and Oil and Gas Extraction	36.99%	35.62%	2.74%	12.33%	10.96%	1.37%	0.00%
19	Construction	72.34%	17.02%	0.00%	10.64%	0.00%	0.00%	0.00%
20	Management of Companies and Enterprises	65.00%	10.00%	5.00%	15.00%	5.00%	0.00%	0.00%
21	Agriculture, Forestry, Fishing and Hunting	55.56%	22.22%	0.00%	16.67%	5.56%	0.00%	0.00%

Result 7 Continued: Cyber-Attacks from 2014 to 2023: Percentage of Attack-Motive for each Industry

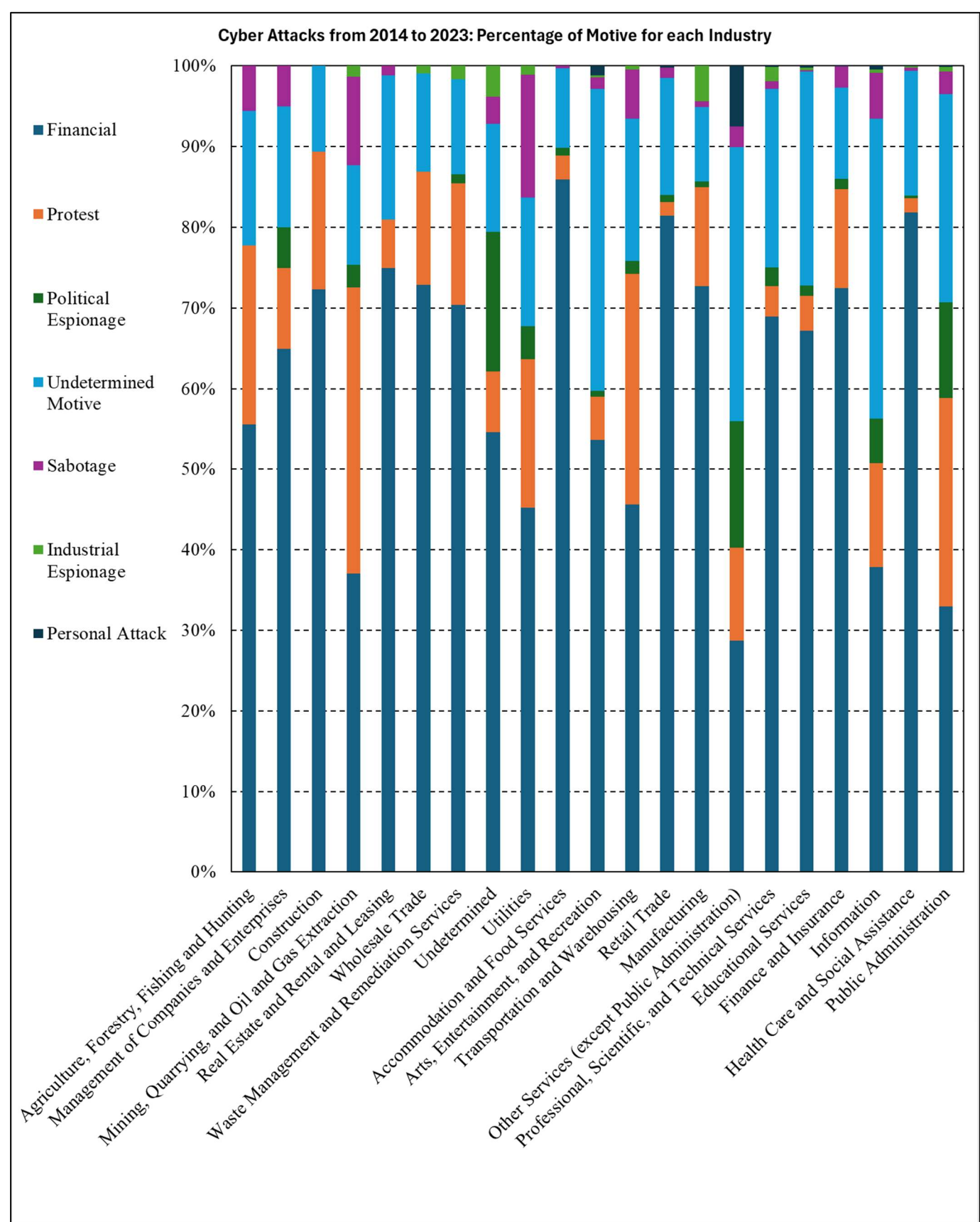


Figure 7: Cyber-Attacks from 2014 to 2023: Percentage of Motive for each Industry

Result 8: Cyber-Attacks from 2014 to 2023 by attack Actor Type

Table 8: Number of Cyber-Attacks from 2014 to 2023 by Actor Type

Attack Actor Type Number	Attack Actor Type	Number of Cyber-Attacks from 2014 to 2023 by actor type	Percentage
1	Criminal	10676	76.03%
2	Hacktivist	1909	13.60%
3	Hacktivist	17	0.12%
4	Hobbyist	191	1.36%
5	Nation-State	803	5.72%
6	Terrorist	30	0.21%
7	Undetermined	415	2.96%

Total = 14041

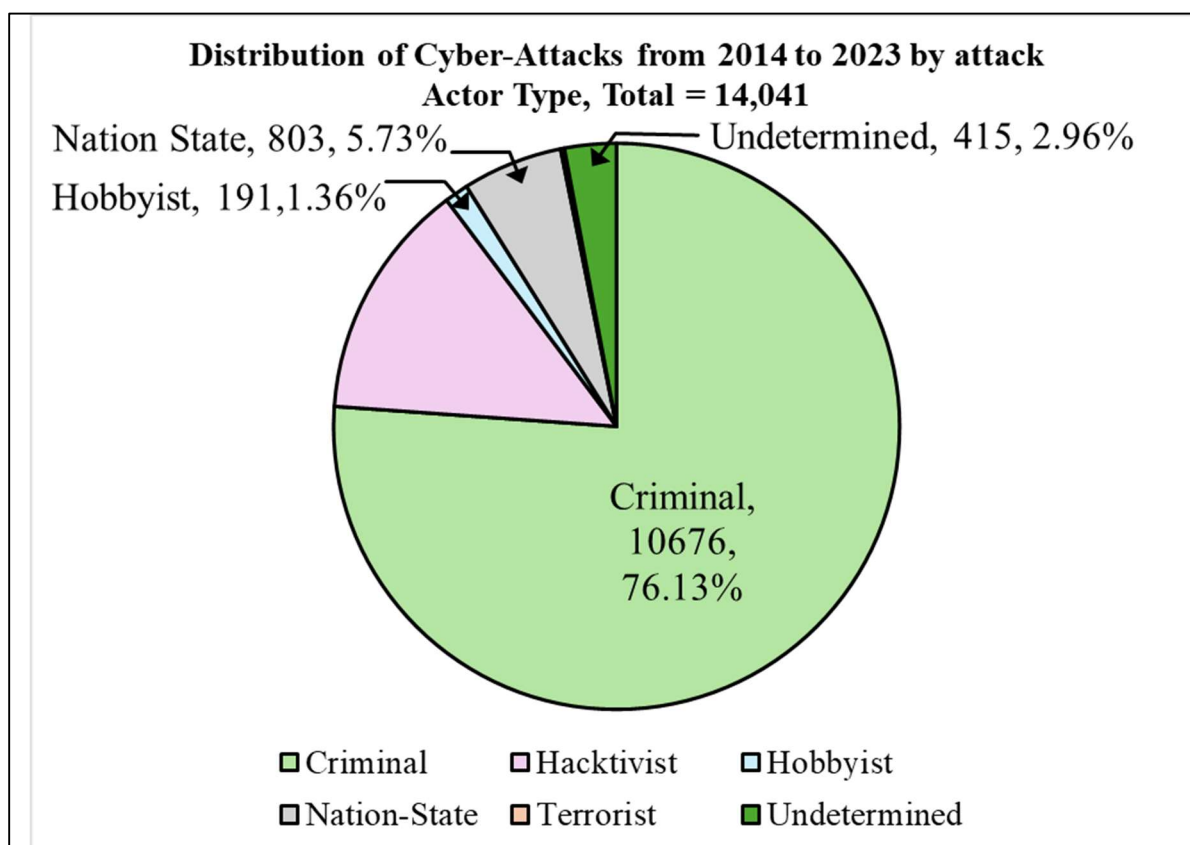


Figure 8: Number of Cyber-Attacks from 2014 to 2023 by attack actor type

Result 9: Cyber-Attacks from 2014 to 2023 by industry and actor-type

Table 9: Number of Cyber-Attacks from 2014 to 2023 by industry and actor-type

Industry Number	Industry Name	Criminal	Hacktivist	Hobbyist	Nation State	Terrorist	Undetermined	Total Industry Cyber-Attacks 2014 to 2023
1	Public Administration	1461	830	24	252	8	65	2640
2	Health Care and Social Assistance	1794	38	3	17	0	13	1865
3	Information	978	213	29	122	14	84	1440
4	Finance and Insurance	1082	168	7	67	0	16	1340
5	Educational Services	1152	67	14	26	1	60	1320
6	Professional, Scientific, and Technical Services	1013	54	12	45	0	45	1169
7	Other Services (except Public Administration)	631	138	27	79	5	56	936
8	Manufacturing	517	79	6	30	0	11	643
9	Retail Trade	426	8	4	12	0	19	469
10	Transportation and Warehousing	272	131	2	33	0	5	443
11	Arts, Entertainment, and Recreation	314	23	55	8	1	19	420
12	Accommodation and Food Services	291	9	1	2	0	2	305
13	Utilities	167	49	4	43	0	7	270
14	Undetermined	141	18	0	45	1	4	209
15	Waste Management and Remediation Services	143	27	0	3	0	5	178
16	Wholesale Trade	87	15	2	1	0	2	107
17	Real Estate and Rental and Leasing	75	6	0	1	0	2	84
18	Mining, Quarrying, and Oil and Gas Extraction	36	23	0	14	0	0	73
19	Construction	39	7	1	0	0	0	47
20	Management of Companies and Enterprises	16	2	0	2	0	0	20
21	Agriculture, Forestry, Fishing and Hunting	13	4	0	1	0	0	18

Result 9 Continued: Cyber-Attacks from 2014 to 2023 by industry and actor-type

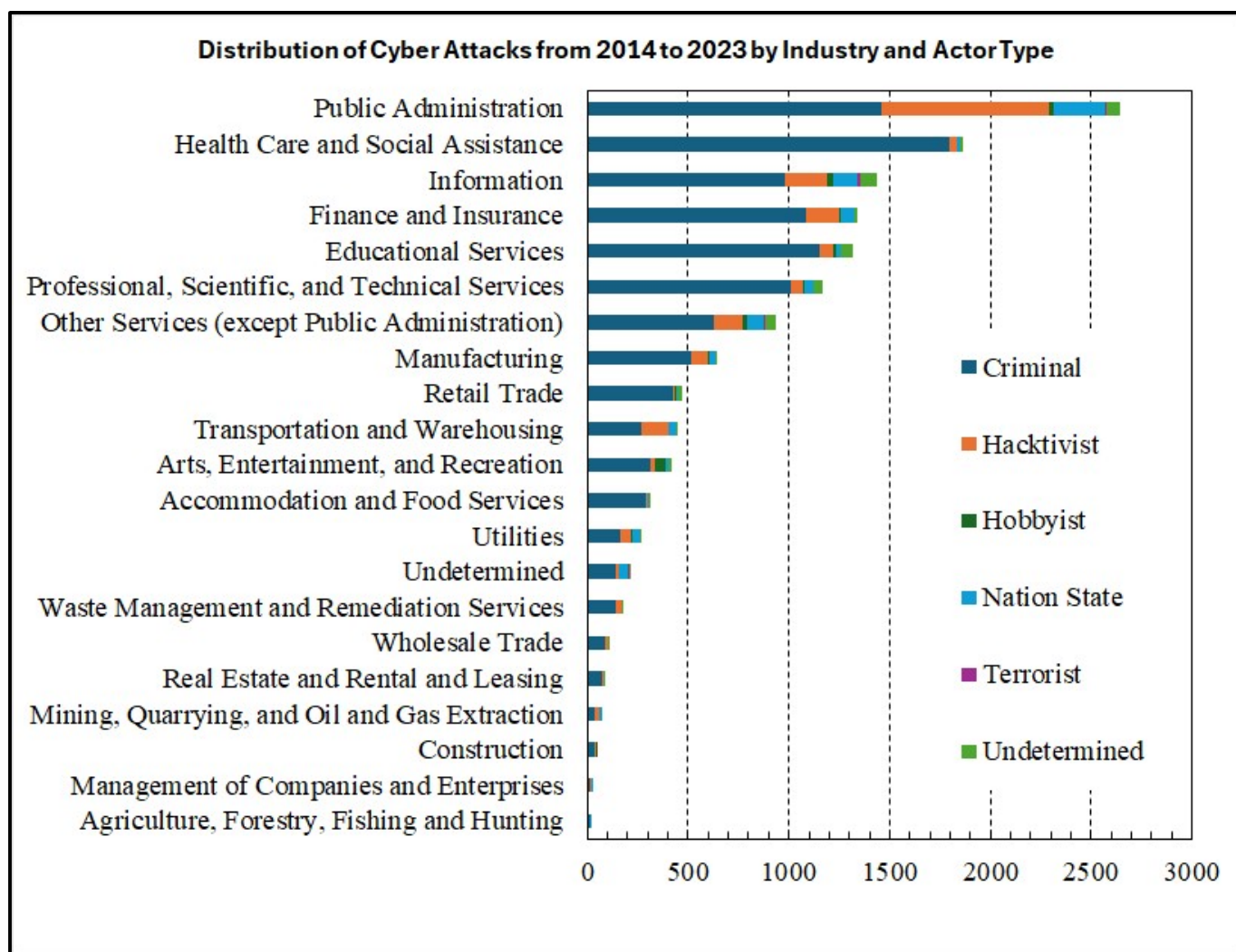


Figure 9: Number of Cyber-Attacks from 2014 to 2023 by industry and Actor-Type

Findings from Result 9:

- Attacks on healthcare and social assistance are almost all done by criminals
- Attacks on public administration are approximately two-thirds done by criminals, followed by hactivists and nation-states.
- As can be seen from the bar graphs, *after removing hactivist, nation-states and terrorists*, health care and social assistance has higher number of cyber-attacks as compared to public administration

Result 10: Cyber-Attacks from 2014 to 2023: Percentage of Actor-Type for each Industry

Table 10: Cyber-Attacks from 2014 to 2023: Percentage of Actor-Type for each Industry

Industry Name	Criminal	Hacktivist	Hobbyist	Nation State	Terrorist	Undetermined
Public Administration	55.34%	31.44%	0.91%	9.55%	0.30%	2.46%
Health Care and Social Assistance	96.19%	2.04%	0.16%	0.91%	0.00%	0.70%
Information	67.92%	14.79%	2.01%	8.47%	0.97%	5.83%
Finance and Insurance	80.75%	12.54%	0.52%	5.00%	0.00%	1.19%
Educational Services	87.27%	5.08%	1.06%	1.97%	0.08%	4.55%
Professional, Scientific, and Technical Services	86.66%	4.62%	1.03%	3.85%	0.00%	3.85%
Other Services (except Public Administration)	67.41%	14.74%	2.88%	8.44%	0.53%	5.98%
Manufacturing	80.40%	12.29%	0.93%	4.67%	0.00%	1.71%
Retail Trade	90.83%	1.71%	0.85%	2.56%	0.00%	4.05%
Transportation and Warehousing	61.40%	29.57%	0.45%	7.45%	0.00%	1.13%
Arts, Entertainment, and Recreation	74.76%	5.48%	13.10%	1.90%	0.24%	4.52%
Accommodation and Food Services	95.41%	2.95%	0.33%	0.66%	0.00%	0.66%
Utilities	61.85%	18.15%	1.48%	15.93%	0.00%	2.59%
Undetermined	67.46%	8.61%	0.00%	21.53%	0.48%	1.91%
Waste Management and Remediation Services	80.34%	15.17%	0.00%	1.69%	0.00%	2.81%
Wholesale Trade	81.31%	14.02%	1.87%	0.93%	0.00%	1.87%
Real Estate and Rental and Leasing	89.29%	7.14%	0.00%	1.19%	0.00%	2.38%
Mining, Quarrying, and Oil and Gas Extraction	49.32%	31.51%	0.00%	19.18%	0.00%	0.00%
Construction	82.98%	14.89%	2.13%	0.00%	0.00%	0.00%
Management of Companies and Enterprises	80.00%	10.00%	0.00%	10.00%	0.00%	0.00%
Agriculture, Forestry, Fishing and Hunting	72.22%	22.22%	0.00%	5.56%	0.00%	0.00%

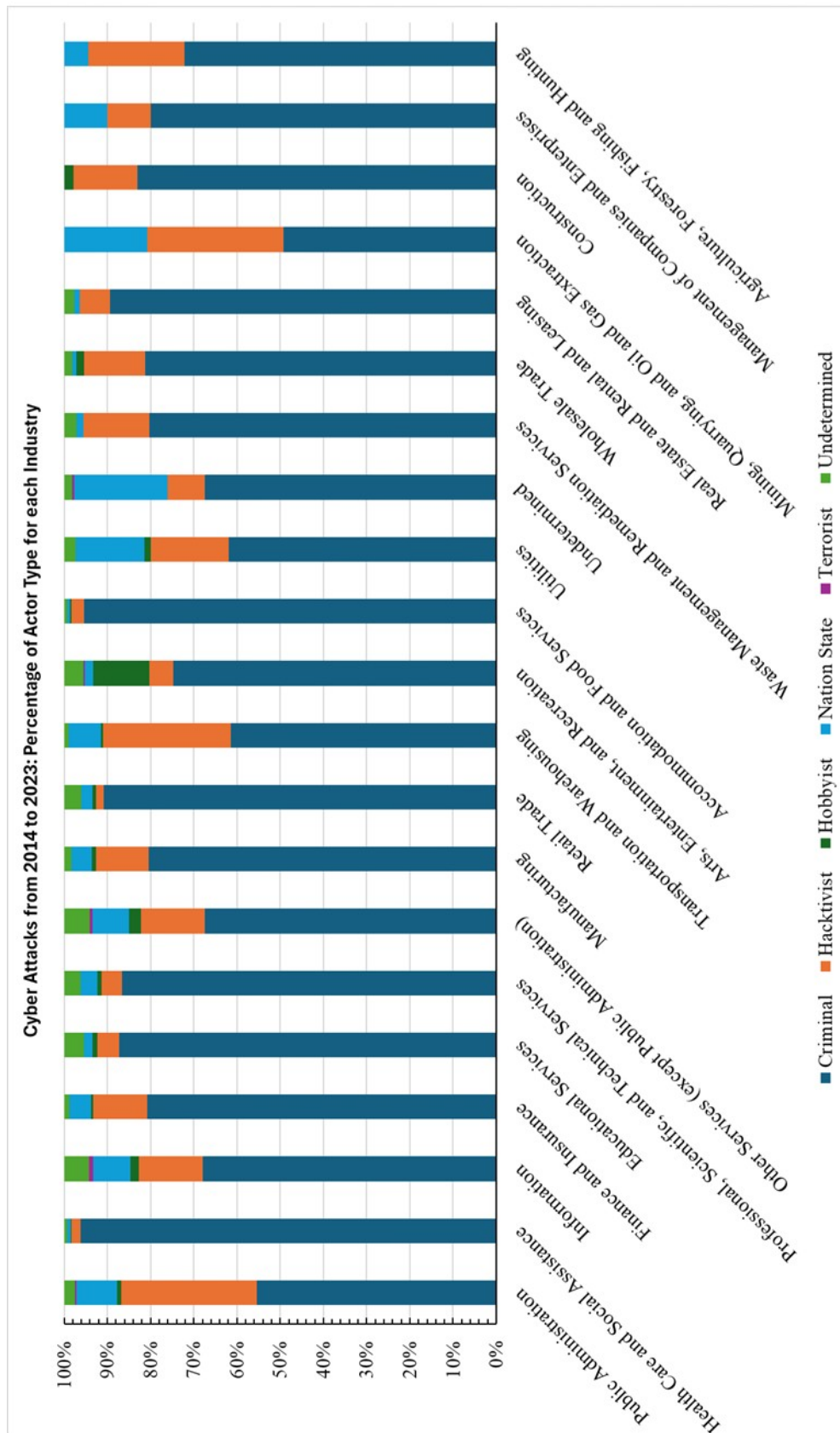


Figure 10: Cyber-Attacks from 2014 to 2023: Percentage of Actor-Type for each Industry

Result 11: Cyber-attacks on three key industries by year

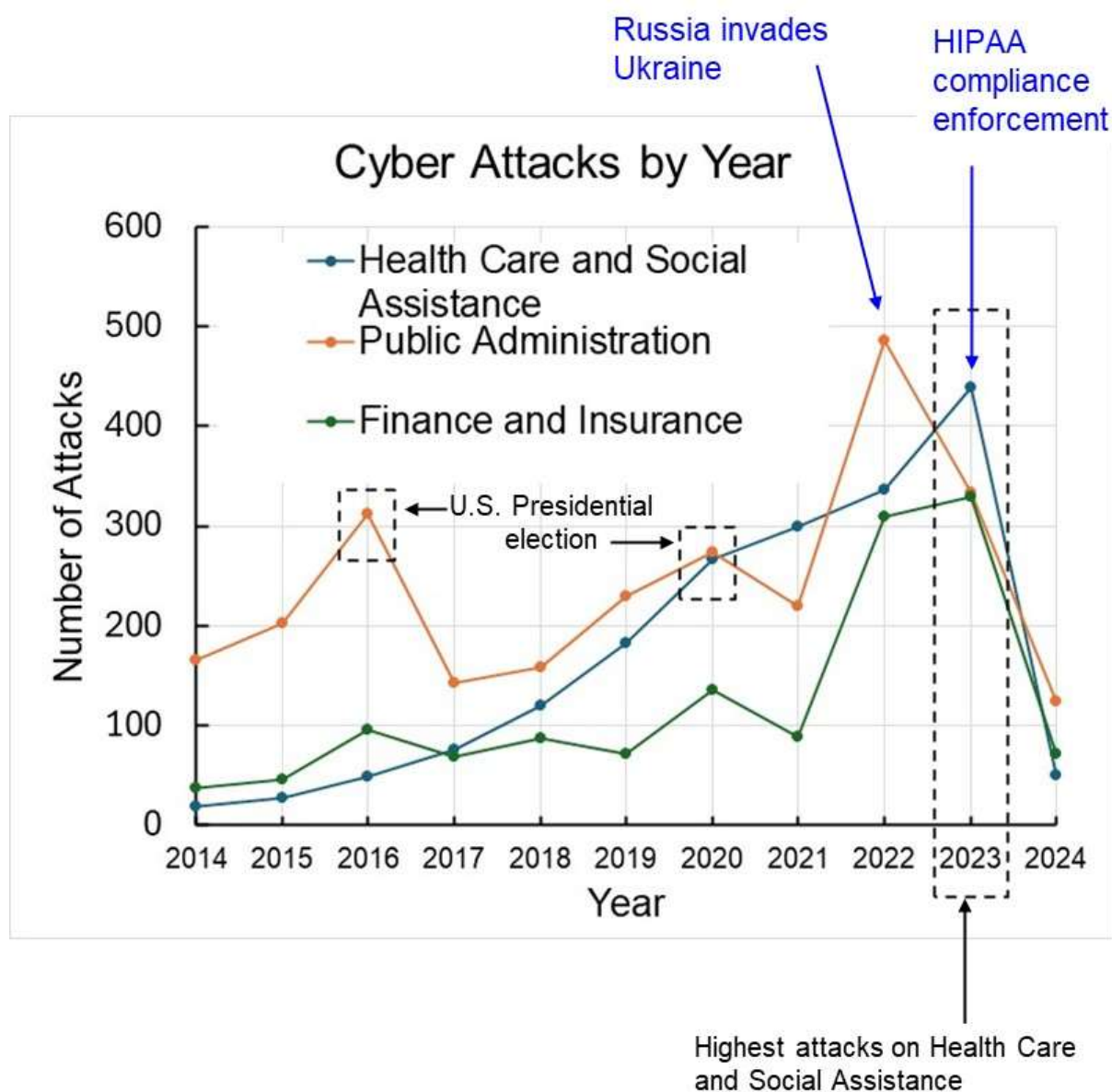


Figure 11: Cyber-attacks on three key industries by year

(Please see next page for findings on Figure 11)

Findings from Figure 11:

Following the scientific method, this analysis studies how cyber-attacks of the three key industries changed by year in order to test validity of the hypothesis over time.

Notable findings from the graph:

1. In general attacks on health care have been steadily increasing over the last decade
2. There are sharp peak in attacks on health care during 2023

Reasoning for 1 and 2: In 2016, a law that electronic Health Records would become legally mandatory by October 2023 was passed. This led to steadily more healthcare data and more facilities accessing data, hence higher attacks, with a sharp peak in 2023

3. Considering three-year periods centered on U.S. Presidential elections, attacks on public administration peaked during the election years 2016 and 2020. The sharper peak in 2016 correlates with the alleged Russian interference in the 2016 election.

Please Note: 2024 data is for a partial year; hence all data points are lower.

Result 12: Cyber-attacks by victim-country GDP

Approach: Based on my results of cyber-attacks over time, this work investigated correlation of cyber-attacks with broader economic and geopolitical factors like GDP and Gini-Coefficient, the results of which are results 12 and result 13.

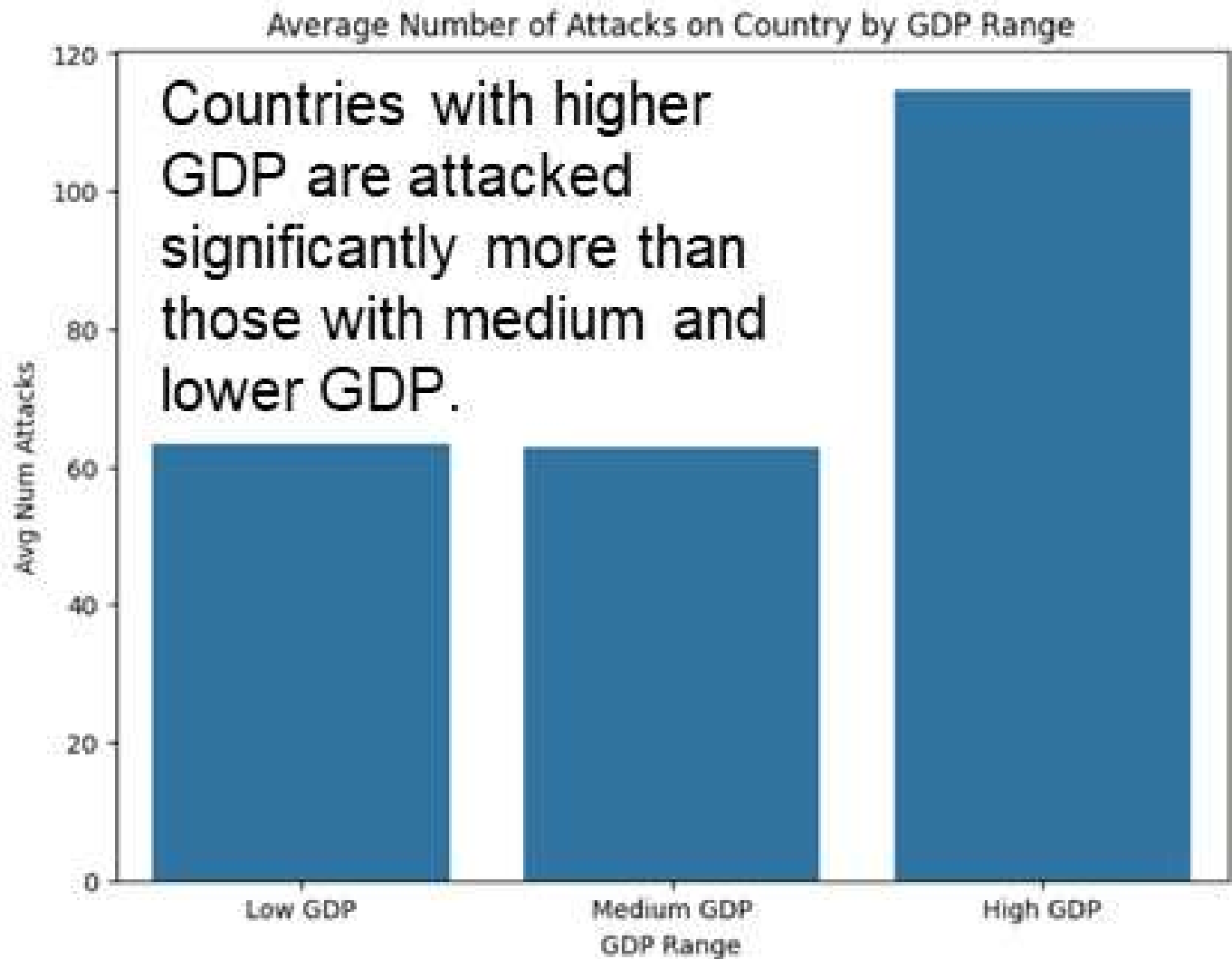


Figure 12: Cyber-attacks by victim-country GDP

Result 13: Cyber-attacks by victim-country Gini Coefficient

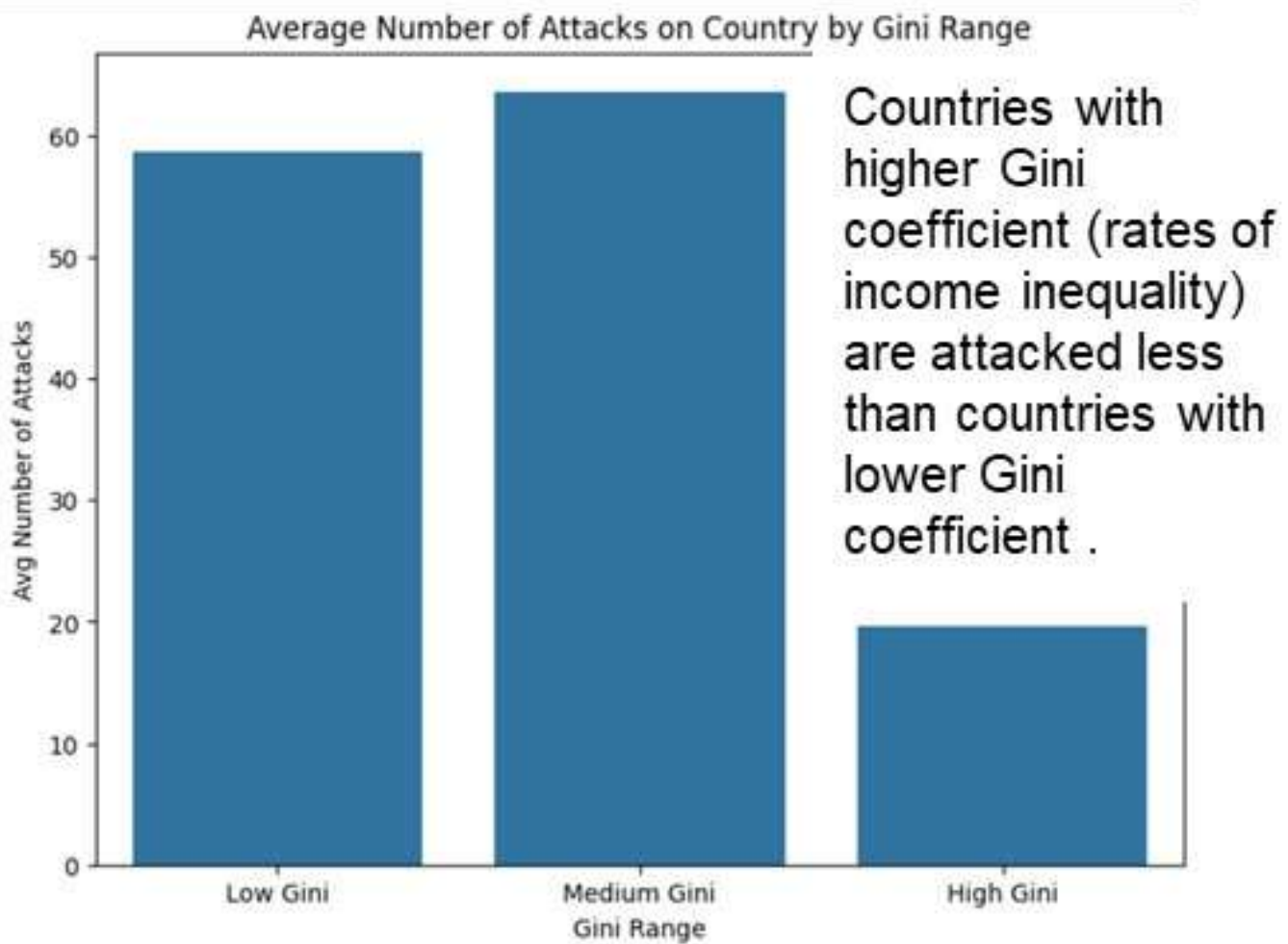


Figure 13: Cyber-attacks by victim-country Gini Coefficient

Result 14: Autocorrelation of cyber-attack time-series vs. lag

Based on my results of cyber-attacks over time, this work then proceeded with research into developing a predictive model for cyber-attacks.

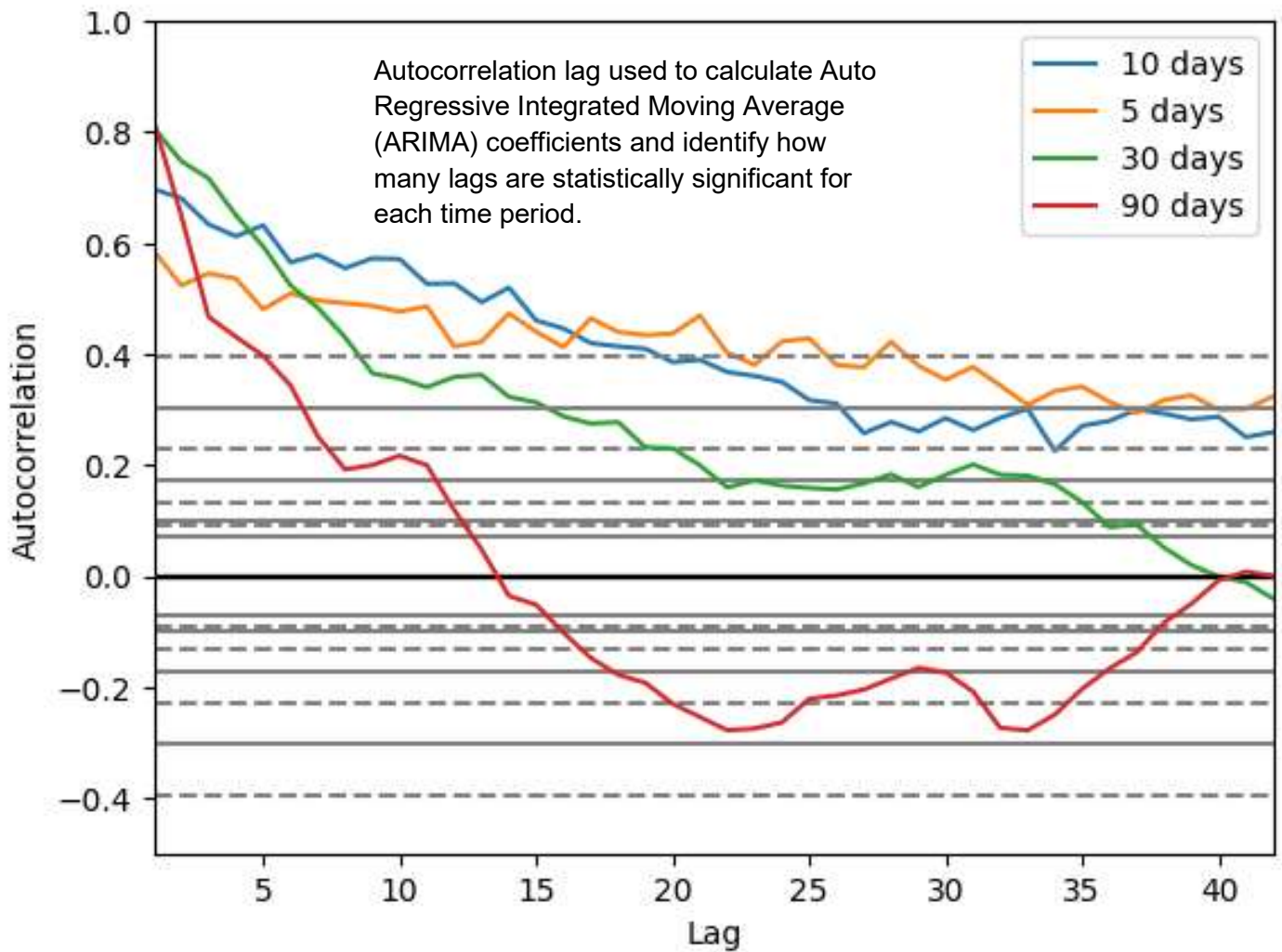


Figure 14: Autocorrelation of cyber-attack time-series vs. lag

Result 15: ARIMA time-series model prediction vs. data

As can be seen in the figure below, the model prediction is close to the cyber-attack data.

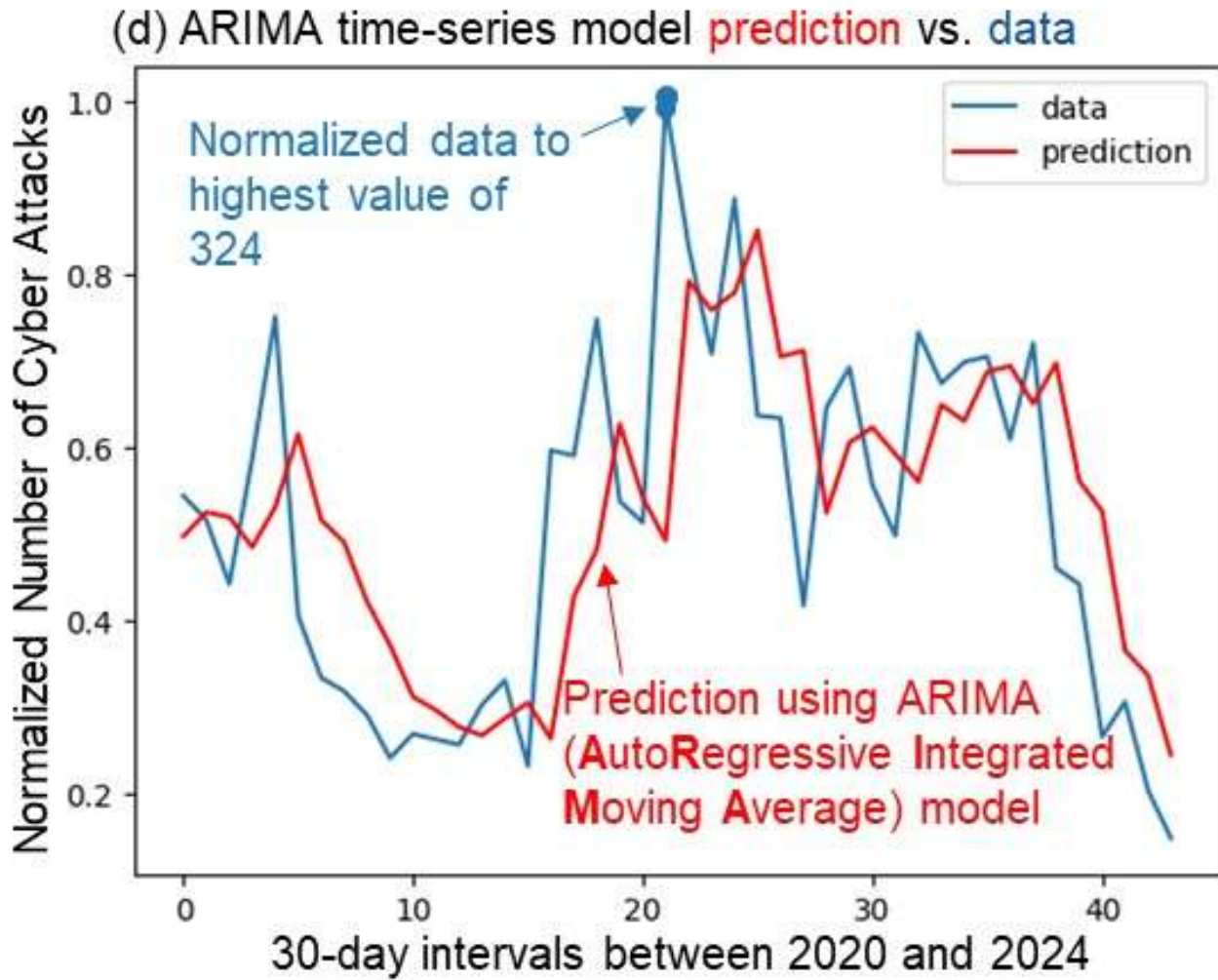


Figure 15: Auto Regressive Integrated Moving Average (ARIMA) time-series model prediction vs. data

Discussion

Consistent with the hypothesis, two out of three of the most recent full years, i.e., 2021 and 2023, attacks on healthcare were highest. The increasing trend of attacks on healthcare and its correlation with the adoption of electronic health records agrees with the basis of the hypothesis.

Considering the whole decade 2014 to 2024, The public administration industry has the highest number of cyber-attacks, followed by health care and social assistance. This finding is close but not exactly consistent with the research hypothesis that health care would have the highest number of attacks. The work then proceeded with a detailed analysis of cyber-attacks and their classification by attack type, motive and actors.

The findings indicate that attacks on public administration, and health care & social assistance differ in terms of these underlying factors: While public administration is found to have an approximately 20% higher number of disruptive attacks than exploitative attacks, the health care and social assistance industry is dominated by exploitative attacks which are approximately 3 times higher than disruptive attacks.

Additionally, in terms of actor type, public administration is attacked by moderately higher (56%) criminals than politically or ideologically motivated nation-states and hacktivists (42%), while attacks on health care and social assistance industries are dominated by criminals (96%).

Then, from a motive perspective, 51% of attacks on the public administration industry had a non-financial motive, whereas approximately 97% of attacks on the health care and social assistance industry had a financial motive.

Furthermore, the motive/actor profiles differ between the exploitative and disruptive attack types.

- The motive of exploitative attacks is mostly financial (78%), while disruptive attacks have a slight majority of non-financial motives (52%).
- Similarly, with actor profile, exploitative attacks have an overwhelming majority of non-political criminal actors (85%) while disruptive attacks also have a large prevalence of criminal actors, though less at 60%

with a larger number of political actors.

Considering cyber-attacks only by non-political actors, i.e., not by nation-states or hacktivists, health care and social assistance is found to have the highest number of cyber-attacks, which supports the hypothesis of this research.

Additionally, countries with higher GDP and countries with greater equality (using Gini coefficient) are attacked more frequently

This is because such countries have more economic goods to capture and are more likely to be targets of political actors due to their greater influence across the world.

Based on my ARIMA time-series model of cyber-attack data, it is clear that trends in cyber-attacks are predictable.

This finding could be used to enhance security during periods of increased vulnerability.

Conclusion

My hypothesis that healthcare would be victim to the greatest number of cyber-attacks was proved accurate in recent years. This is due to the only recent adoption of Electronic Health records as per legislation passed in 2016, coming into action by 2023 (wherein there was a clear spike in cyber-attacks on the Healthcare Industry) When observing the whole decade. Public Administration was seen as having the greatest number of cyber-attacks.

However, when considering only either exploitative attacks or those with financial motive, healthcare still has the greatest number of cyber-attacks.

This is due to the large concentration of personal data, recurrent access to the data by numerous individuals, and potentially outdated systems.

The only industry that surpassed health care in number of cyber was public administration due to a large number of politically motivated disruptive attacks.

This research identifies the industry with the greatest number of cyber-attacks and criminological and economic factors unlike prior studies found in my literature review:

- Politically motivated disruptive attacks resulted in public administration having the greatest number of cyber attacks
- Financially motivated attacks resulted in a large number of attacks on the healthcare industry
- Using GDP, countries with higher economic resources are attacked more frequently as attackers see a greater reward and that they are more influential on the global scene resulting in more enemies and politically motivated attacks.
- Using Gini coefficient, countries with less inequality are attacked more frequently as those countries generally have greater overall prosperity to exploit.

My time series analysis finds that there are predictable trends in cyber-attacks that can help to enhance security at high-risk moments.

Application-wise, my research can help to identify the where and why of cyber-attacks such that government policy and private cybersecurity work can focus on solving the problems where they exist.

Also, my work demonstrates the efficacy of using data science to recognize patterns in records of social science events.

Recommendations

- What problems arose during the experiment?

An attempt was initially made to do a linear regression model to predict cyber-attacks based on GDP and Gini coefficient. However, this model did not fit the data. Hence, a different path was pursued to model the cyber-attacks using the time-series Auto Regressive Integrated Moving Average (ARIMA) model.

- What would you have done differently?

This project could be conducted differently by analyzing specific relations using more algorithmic and statistical models rather than focusing on visual representation. For example, I could analyze the attacks on the healthcare sector versus the adoption of electronic health records. I might also have analyzed the attacks on the public administration sector versus certain measures of political or economic stability. For analyzing motives, actor-types, and event-types, I would analyze their relationships more in depth; for example, to identify what are the motives per each actor-type.

- If you were to continue this project, what would you do next

A continuation of this project would analyze the size of each attack and its effects quantitatively. This could be done by looking at monetary impacts on victims, the number of victims' data that is affected, or how long the effects last. Such analysis would ensure that the analysis is not skewed by many smaller attacks on certain industries, allowing a clearer finding by only including attacks that are of a certain size. Also, this project could find causes in specific political dynamics between companies by examining the relations between the actor and victim states.

Bibliography

- Anderson, R. and Moore, T. (2006). The Economics of Information Security. *Science*, [online] 314(5799), pp.610–613. Available at: <https://www.jstor.org/stable/20031627> [Accessed 23 Jul. 2024].
- Bhosale, K.S., Nenova, M. and Iliev, G. (2021). A study of cyber attacks: In the healthcare sector. *IEEE Xplore*, [online] pp.1–6. doi:<https://doi.org/10.1109/Lighting49406.2021.9598947>.
- Blank, S., Perkovich, G. and Levite, A. (n.d.). *Cyber War and Information War à la Russe*. [online] Available at: https://carnegie-production-assets.s3.amazonaws.com/static/files/GUP_Perkovich_Levite_UnderstandingCyberConflict_Ch5.pdf [Accessed 1 Mar. 2025].
- Center for International and Security Studies at Maryland. (2024). *Cyber Events Database*. [data set] Available at: <https://cissm.umd.edu/cyber-events-database>.
- CISA (2024). *Russian Military Cyber Actors Target US and Global Critical Infrastructure* | CISA. [online] Cybersecurity and Infrastructure Security Agency CISA. Available at: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa24-249a> [Accessed 28 Feb. 2025].
- Coventry, L. and Branley, D. (2018). Cybersecurity in healthcare: A narrative review of trends, threats and ways forward. *Maturitas*, [online] 113(113), pp.48–52. doi:<https://doi.org/10.1016/j.maturitas.2018.04.008>.
- CyberPeace Institute (n.d.). *Cyberattacks Impact and Harm on the Public administration sector* | CyberPeace Institute. [online] cyberconflicts.cyberpeaceinstitute.org. Available at: <https://cyberconflicts.cyberpeaceinstitute.org/impact/sectors/public-administration> [Accessed 21 Sep. 2024].
- Electronic Privacy Information Center (2024). *Medical Records*. [online] Epic.org. Available at: https://archive.epic.org/privacy/consumer/med_record.html.
- Murray, C. and Srivastava, M. (2022). How Conti ransomware group crippled Costa Rica — then fell apart. *Financial Times*. [online] 9 Jul. Available at: <https://www.ft.com/content/9895f997-5941-445c-9572-9cef66d130f5>.
- Myrick, K.L., McNeal, M. and DeFrances, C. (2022). QuickStats: Percentage of Office-Based Physicians Using Telemedicine Technology, by Specialty — United States, 2019 and 2021. *Morbidity and Mortality Weekly*

Report (MMWR), [online] 71. doi:<https://doi.org/10.15585/mmwr.mm7149a6>.

Office of the Director of National Intelligence (2024). *Recent Cyber Attacks on US Infrastructure Underscore Vulnerability of Critical US Systems*. [online] Available at:
https://www.dni.gov/files/CTIIC/documents/products/Recent_Cyber_Attacks_on_US_Infrastructure_Underscore_Vulnerability_of_Critical_US_Systems-June2024.pdf.

Sophos (2024). *Cybersecurity as a Service Delivered | Sophos*. [online] Available at:
<https://www.sophos.com/en-us/press/press-releases/2024/09/two-thirds-healthcare-organizations-hit-ransomware-four-year-high>.

World Bank (2025), Poverty and Inequality Platform (version 20240627_2017_01_02_PROD) [data set].
pip.worldbank.org. Available at: <https://pip.worldbank.org/poverty-calculator>

Appendix 1: Google Colab python

Code

```
# -*- coding: utf-8 -*-
```

```
"""newCyberPaper
```

Automatically generated by Colab.

Original file is located at

https://colab.research.google.com/drive/1ThXrjvK0lxD_CQ6YjgvV7VEjMUwJWke7

Setting Up

```
"""
```

```
# Commented out IPython magic to ensure Python compatibility.
```

```
import pandas as pd
```

```
import numpy as np
```

```
import seaborn as sns
```

```
import plotly
```

```
import matplotlib.pyplot as plt
```

```
import plotly.express as px
```

```
from google.colab import drive
```

```
import numpy as np
```

```
import plotly.graph_objects as go
```

```
drive.mount('/content/drive')
```

```
# %cd /content/drive/My Drive/GoogColab/effectsBreach/data
```

```
df = pd.read_csv('mainData.csv', encoding='latin-1')
```

```
"""# Data Processing"""
```

```
## event_counts stores cyber event types over all industries
```

```
event_counts = df.groupby('event_type').size().reset_index(name='counts')
```

```
event_counts
```

```
## motive_counts stores motive over all industries
```

```
motive_counts = df.groupby('motive').size().reset_index(name='counts')
```

```
motive_counts
```

```
## actor_counts stores actor types over all industries
```

```
actor_counts = df.groupby('actor_type').size().reset_index(name='counts')
```

```
actor_counts
```

```

## monthly counts stores month, event_type, and industry
monthly_counts = df.groupby(['month', 'event_type', 'industry']).size().reset_index(name='count')

## dfEventType stores occurrences of each event type within each industry
dfEventType = df.groupby('industry')['event_type'].value_counts().unstack(fill_value=0)

# Rename columns for clarity
dfEventType = dfEventType.rename(columns={
    'Undetermined': 'undetermined_eventType_count',
    'Mixed': 'mixed_eventType_count',
    'Exploitive': 'exploitive_eventType_count',
    'Disruptive': 'disruptive_eventType_count'
})

# Calculate total counts for each industry
dfEventType['industry_counts'] = dfEventType.sum(axis=1)

# Reset index to make 'industry' a regular column
dfEventType = dfEventType.reset_index()

# Sort dfEventType by industry_counts in ascending order
dfEventType = dfEventType.sort_values(by='industry_counts', ascending=True)
dfEventType.head()

## dfActorType stores occurrences of each actor_type within each industry
dfActorType = df.groupby('industry')['actor_type'].value_counts().unstack(fill_value=0)
dfActorType['industry_counts'] = dfActorType.sum(axis=1)
dfActorType = dfActorType.reset_index()
dfActorType = dfActorType.sort_values(by='industry_counts', ascending=True)
dfActorType.head()

## dfMotive stores occurrences of each motive type within each industry
dfMotive = df.groupby('industry')['motive'].value_counts().unstack(fill_value=0)
dfMotive['industry_counts'] = dfMotive.sum(axis=1)
dfMotive = dfMotive.reset_index()
dfMotive = dfMotive.sort_values(by='industry_counts', ascending=True)
dfMotive.rename(columns = {'industry': 'Sector Title', 'Undetermined': 'Undetermined Motive'}, inplace = True)
dfMotive.head()

## combine dfEventType, dfActorType, and dfMotive type into final merged df

dfActorType = dfActorType.sort_values(by='industry_counts', ascending=True)
dfEventType = dfEventType.sort_values(by='industry_counts', ascending=True)
dfMotive = dfMotive.sort_values(by='industry_counts', ascending=True)
dfActorType.rename(columns = {'industry': 'Sector Title'}, inplace = True)
dfEventType.rename(columns = {'industry': 'Sector Title'}, inplace = True)
dfMotive.rename(columns = {'industry': 'Sector Title'}, inplace = True)

merged_df = pd.merge(dfEventType, dfActorType, on='Sector Title', how='outer')
final_merged_df = pd.merge(merged_df, dfMotive, on='Sector Title', how='outer')
final_merged_df = final_merged_df.sort_values(by='industry_counts_y', ascending=True)

```

```

final_merged_df.head()

## create lists of values for certain columns for use throughout
listEvent = [ 'disruptive_count',
'exploitive_count',
'mixed_count',
'undetermined_count']

listActor = ['Criminal',
'Hactivist',
'Hobbyist',
'Nation-State',
'Terrorist',
'Undetermined']

listMotive = ['Financial',
'Industrial-Espionage',
'Personal Attack',
'Political-Espionage',
'Protest',
'Protest,Financial',
'Protest;Political-Espionage',
'Sabotage',
'Undetermined Motive']

industry_types = [ "Agriculture, Forestry, Fishing and Hunting"
"Mining",
"Utilities",
"Construction",
"Manufacturing",
"Wholesale Trade",
"Retail Trade",
"Transportation and Warehousing",
"Information",
"Finance and Insurance",
"Real Estate Rental and Leasing",
"Professional, Scientific, and Technical Services",
"Management of Companies and Enterprises",
"Administrative and Support and Waste Management and Remediation Services",
"Educational Services",
"Health Care and Social Assistance",
"Arts, Entertainment, and Recreation",
"Accommodation and Food Services",
"Other Services (except Public Administration)",
"Public Administration"]

## create industryDF which stores event_date, industry, and counts
## create timeDF which stores event_date and counts

df['event_date'] = pd.to_datetime(df['event_date'], format = 'mixed')

```

```

event_types = ['Undetermined', 'Disruptive', 'Exploitive', 'Mixed']
filtered_df = df[df['event_type'].isin(event_types)]

timeDf = filtered_df.groupby([pd.Grouper(key='event_date', freq='M')]).size().reset_index(name='counts')

# Convert event_date to datetime
df['event_date'] = pd.to_datetime(df['event_date'])

# Filter rows based on event_type values
event_types = ['Undetermined', 'Disruptive', 'Exploitive', 'Mixed']
filtered_df = df[df['event_type'].isin(event_types)]

# Group by year, month, and event_type, then count occurrences
grouped_df = filtered_df.groupby([pd.Grouper(key='event_date', freq='M'),
'event_type']).size().reset_index(name='counts')

filtered_df = df[df['industry'].isin(industry_types)]

# Group by year, month, and event_type, then count occurrences
industryDf = filtered_df.groupby([pd.Grouper(key='event_date', freq='ME'),
'industry']).size().reset_index(name='counts')

# df_health contains only the data of attacks on the Health Care and Social Assistance industr
df_health = df[df['industry'] == 'Health Care and Social Assistance']

# df_publicAdmin contains only data of attacks on the Public Adminsitration Industry
df_publicAdmin = df[df['industry'] == 'Public Administration']

"""# Over All Industries

# Distribution of Cyber Events over All Industries
"""

import plotly.express as px
import plotly.graph_objects as go

fig = px.bar(df, y=df['industry'].value_counts().index, x=df['industry'].value_counts().values,
            title='Distribution of Cyber Attacks Across Industries')

fig.update_layout(
    width=800,
    height=600,
    yaxis_title="Industry",
    xaxis_title="Number of Attacks"
)

fig.update_layout(font=dict(size=14))
fig.show()
# this code graphs the number of cyber attacks per industry using plotly

```

```

industry_counts = df['industry'].value_counts().sort_values(ascending=True)
plt.figure(figsize=(10, 8))
industry_counts.plot(kind='bar')
plt.xlabel('Number of Attacks')
plt.ylabel('Industry')
_ = plt.title('Figure 1: Distribution of Cyber Attacks Across Industries')
# this code graphs the number of cyber attacks per industry using Matplotlib

""""# Cyber Event Types over All Industries""""

fig = px.bar(event_counts,
             y='event_type',
             x='counts',
             orientation='h',
             color_discrete_sequence=px.colors.qualitative.Dark2,
             labels={'event_type': 'Event Type', 'counts': 'Number of Attacks'},
             title='Distribution of Cyber Attacks by Event Type Across All Industries')

fig.update_layout(
    yaxis_title='Event Type',
    xaxis_title='Number of Attacks',
    showlegend=False,
    plot_bgcolor='white'
)
fig.update_layout(font=dict(size=14))
fig.show()
# this code graphs the distribution of cyber attack event types across all industries using plotly

fig, ax = plt.subplots(figsize=(12, 8))

ax.barh(event_counts['event_type'], event_counts['counts'])

ax.set_xlabel('Number of Attacks')
ax.set_ylabel('Event Type')
ax.set_title('Distribution of Cyber Attacks by Event Type Across All Industries')

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# this code graphs the distribution of cyber attack event types across all industries using Matplotlib

fig = px.bar(dfEventType, y = "Sector Title",
             x = ["disruptive_eventType_count", "exploitive_eventType_count",
                  "mixed_eventType_count", "undetermined_eventType_count"],
             title = "Distribution of Cyber Events and Types Across Industries" )
fig.update_layout(font=dict(size=14))
fig.show()
# graph cyber events across all industry as well as coloring by event-type

```



```

dfEventType[['disruptive_eventType_count', 'exploititive_eventType_count', 'mixed_eventType_count',
'undetermined_eventType_count']].plot(kind='bar', stacked=True, figsize=(12, 6))
plt.title('Distribution of Cyber Events and Types Across Industries')
plt.xlabel('Industry')
plt.ylabel('Number of Cyber Attacks')
plt.xticks(range(len(dfEventType)), dfEventType['Sector Title'], rotation=90)
_ = plt.legend(title='Event Types')
#plt.show()

import plotly.graph_objects as go

fig = go.Figure(data=go.Heatmap(
    z=dfEventType[["disruptive_eventType_count", "exploititive_eventType_count", "mixed_eventType_count",
"undetermined_eventType_count"]].values,
    x=["disruptive_eventType_count", "exploititive_eventType_count", "mixed_eventType_count",
"undetermined_eventType_count"],
    y=dfEventType["Sector Title"], colorscale="tempo"
))

fig.update_layout(title='Heatmap of Event Types by Industry')
fig.update_layout(font=dict(size=14))
fig.show()
# graph heatmap of industry vs event_type for cyber events

fig = px.pie(event_counts, values='counts', names='event_type', title = 'Pie Chart of Cyber Event Types')
fig.update_layout(font=dict(size=14))
fig.show()

""""# Motives of Cyber Event Types over all Industries""""

fig = px.pie(motive_counts, values='counts', names='motive', title = 'Pie Chart of Motives')
fig.update_layout(font=dict(size=14))
fig.show()
# pie chart of motives over all industries

fig = px.bar(dfMotive, y = "Sector Title",
    x = listMotive,
    title = "Distribution of Motive Types Across Industries" )

fig.show()
# graph cyber events across all industry as well as coloring by event-type

fig = go.Figure(data=go.Heatmap(
    z=final_merged_df[listMotive].values,
    x=listMotive,
    y=final_merged_df["Sector Title"], colorscale="tempo"
))

fig.update_layout(title='Heatmap of Motive Types by Sector Title')

```

```

fig.show()
# graph heatmap of industry vs event_type for cyber events

fig = px.bar(motive_counts, y = 'motive',
             x = 'counts',
             orientation = 'h',
             color_discrete_sequence=px.colors.qualitative.Dark2,
             labels={'motive': 'Motive Counts', 'counts': 'Number of Attacks'},
             title='Distribution of Cyber Attacks by Motive Type Across All Industries')

fig.update_layout(
    yaxis_title='Event Type',
    xaxis_title='Number of Attacks',
    showlegend=False,
    plot_bgcolor='white'
)

fig.update_layout(font=dict(size=14))
fig.show()
# this code graphs the distribution of cyber attack actor types across all industries

""""# Actor Types of Cyber Events over all Industries""""

fig = px.pie(actor_counts, values='counts', names='actor_type', title = 'Pie Chart of Actor Types')
fig.update_layout(font=dict(size=14))
fig.show()

import plotly.graph_objects as go

fig = go.Figure(data=go.Heatmap(
    z=dfActorType[listActor].values,
    x=listActor,
    y=dfActorType["Sector Title"], colorscale="tempo"
))

fig.update_layout(title='Heatmap of Actor Types by Industry')

fig.show()
# graph heatmap of industry vs event_type for cyber events

fig = px.bar(dfActorType, y = "Sector Title",
             x = listActor,
             title = "Distribution of Actor Types Across Industries" )

fig.show()
# graph cyber events across all industry as well as coloring by actor type

import matplotlib.pyplot as plt
import pandas as pd

```

```

dfActorType[['Criminal', 'Hacktivist', 'Hobbyist', 'Nation-State', 'Terrorist', 'Undetermined']].plot(kind='bar',
stacked=True, figsize=(12, 6))
plt.title('Distribution of Actor Types Across Industries')
plt.xlabel('Industry')
plt.ylabel('Number of Cyber Attacks')
plt.xticks(range(len(dfActorType)), dfActorType['Sector Title'], rotation=90)
_ = plt.legend(title='Actor Type')
#plt.show()

```

```

fig = px.bar(actor_counts, y = 'actor_type',
             x = 'counts',
             orientation = 'h',
             color_discrete_sequence=px.colors.qualitative.Dark2,
             labels={'actor_type': 'Actor Type', 'counts': 'Number of Attacks'},
             title='Distribution of Cyber Attacks by Actor Type Across All Industries')

```

```

fig.update_layout(
    yaxis_title='Event Type',
    xaxis_title='Number of Attacks',
    showlegend=False,
    plot_bgcolor='white'
)

```

```

fig.update_layout(font=dict(size=14))
fig.show()
# this code graphs the distribution of cyber attack actor types across all industries

```

""""# Visualizations Over Time""""

```

# Convert event_date to datetime
df['event_date'] = pd.to_datetime(df['event_date'], format = 'mixed')

```

```

# Filter rows based on event_type values
event_types = ['Undetermined', 'Disruptive', 'Exploitive', 'Mixed']
filtered_df = df[df['event_type'].isin(event_types)]

```

```

# Group by year, month, and event_type, then count occurrences
grouped_df = filtered_df.groupby([pd.Grouper(key='event_date', freq='M'),
'event_type']).size().reset_index(name='counts')

```

```

# Plot the data
plt.figure(figsize=(12, 6))

```

```

for event_type in event_types:
    subset_df = grouped_df[grouped_df['event_type'] == event_type]
    plt.plot(subset_df['event_date'], subset_df['counts'], label=event_type)

```

```

plt.xlabel('Time (Year-Month)')
plt.ylabel('Frequency Count')
plt.title('Event Type Frequency Over Time')
plt.legend()

```

```

plt.show()
# this code graphs each cyber event frequency over time in terms of month(s)

fig = px.line(grouped_df, x='event_date', y='counts', color='event_type', title='Event Type Frequency Over Time')

fig.update_xaxes(title_text='Time (Year-Month)')
fig.update_yaxes(title_text='Frequency Count')
fig.update_layout(font=dict(size=14))

fig.show()
# this code graphs each cyber event frequency over time in terms of month(s)

fig = px.line(timeDf, x='event_date', y='counts', title='All Over Time')

fig.update_xaxes(title_text='Time (Year-Month)')
fig.update_yaxes(title_text='Frequency Count')

fig.update_layout(font=dict(size=14))
fig.show()
# this code graphs each cyber event frequency over time in terms of month(s)

from plotly.express import scatter
fig.update_layout(font=dict(size=14))
scatter(data_frame=grouped_df, x='event_date', y='counts', color='event_type', trendline='lowess',
trendline_scope='trace')

fig = px.line(industryDf, x='event_date', y='counts', color='industry', title='Cyber Attacks over Time by Industry')

fig.update_xaxes(title_text='Time (Year-Month)')
fig.update_yaxes(title_text='Frequency Count')
fig.update_layout(font=dict(size=14))
fig.show()
# this code graphs each cyber event frequency over time in terms of month(s)

fig = px.bar(industryDf, x='event_date', y='counts', color='industry', title='Cyber Attacks over Time by Industry')
fig.update_layout(font=dict(size=14))
fig.show()

fig = px.bar(monthly_counts, x='month', y='count', color='industry', title='Cyber Attacks over Time by Industry')
fig.update_layout(font=dict(size=14))
fig.show()

"""# Visualization of Healthcare"""

event_counts = df_health.groupby('event_type').size().reset_index(name='counts')

```

```
fig = px.bar(event_counts,
             y='event_type',
             x='counts',
             orientation='h',
             color_discrete_sequence=px.colors.qualitative.Dark2,
             labels={'event_type': 'Event Type', 'count': 'Number of Attacks'},
             title='Distribution of Cyber Attacks by Event Type in Health Care and Social Assistance Cyber Events')
```

```
fig.update_layout(
    yaxis_title='Event Type',
    xaxis_title='Number of Attacks',
    showlegend=False,
    plot_bgcolor='white'
)
```

```
fig.show()
# graphs the distribution of cyber attacks by event type in the Health Care and Social Assistance Industry
```

```
""""# Visualization of Public Administration""""
```

```
import plotly.express as px
```

```
event_counts = df_publicAdmin.groupby('event_type').size().reset_index(name='counts')
```

```
fig = px.bar(event_counts,
             y='event_type',
             x='counts',
             orientation='h',
             color_discrete_sequence=px.colors.qualitative.Dark2,
             labels={'event_type': 'Event Type', 'count': 'Number of Attacks'},
             title='Distribution of Cyber Attacks by Event Type in Public Administration Cyber Events')
```

```
fig.update_layout(
    yaxis_title='Event Type',
    xaxis_title='Number of Attacks',
    showlegend=False,
    plot_bgcolor='white'
)
```

```
fig.show()
# graphs the distribution of cyber attacks by event type in the Public Administration Industry
```

```
""""# Stats""""
```

```
df_2023 = df[pd.to_datetime(df['event_date']).dt.year == 2023]
```

```
# Group by industry_code and count cyber events in 2024
cyber_events_2023 = df_2023.groupby('industry')['event_type'].count().reset_index()
cyber_events_2023.rename(columns={'event_type': 'cyber_events_2023'}, inplace=True)
```

```
df_2022 = df[pd.to_datetime(df['event_date']).dt.year == 2022]
```

```

cyber_events_2022 = df_2022.groupby('industry')['event_type'].count().reset_index()
cyber_events_2022.rename(columns = {'event_type': 'cyber_events_2022'}, inplace=True)
cyber_events_2022.rename(columns = {'industry': 'Sector Title'}, inplace = True)

df_2021 = df[pd.to_datetime(df['event_date']).dt.year == 2021]
cyber_events_2021 = df_2021.groupby('industry')['event_type'].count().reset_index()
cyber_events_2021.rename(columns = {'event_type': 'cyber_events_2021'}, inplace=True)
cyber_events_2021.rename(columns = {'industry': 'Sector Title'}, inplace = True)

df_2020 = df[pd.to_datetime(df['event_date']).dt.year == 2020]
cyber_events_2020 = df_2020.groupby('industry')['event_type'].count().reset_index()
cyber_events_2020.rename(columns = {'event_type': 'cyber_events_2020'}, inplace=True)
cyber_events_2020.rename(columns = {'industry': 'Sector Title'}, inplace = True)

df_2020.describe()

df_2021.describe()

df_2022.describe()

df_2023.describe()

merged_df.describe()

## country analysis
## correlate vs gini coefficient
## world happiness report
## y # attack vs x #factors including gini and stuff in world happiness report
## remember year
## actor
## target
## unemployment

"""# Country"""

## graph top ten actor Country vertical bar chart

##

"""# Setting up for Gini Coefficient"""

## gini coefficient
## data: https://www.kaggle.com/datasets/willianoliveiragibin/gini-coefficient-after-tax?resource=download
giniDf = pd.read_csv('gini.csv', encoding='latin-1')
giniDf2 = pd.read_csv('gini_by_country.csv', encoding='latin-1')

giniDf2

actorStateCts = df.groupby('actor_country').size().reset_index(name='counts')
actorStateCts.sort_values(by='counts', ascending=False, inplace=True)
actorStateCts

```

```

fig = px.bar(actorStateCts,
             y='actor_country',
             x='counts',
             orientation='h',
             color_discrete_sequence=px.colors.qualitative.Dark2,
             labels={'actor_country': 'Actor Country', 'counts': 'Number of Attacks'},
             title='Distribution of Cyber Attacks by Actor Country')

fig.update_layout(
    yaxis_title='Actor Country',
    xaxis_title='Number of Attacks',
    showlegend=False,
    plot_bgcolor='white'
)

fig.show()

actorStateCtsDrop = actorStateCts.drop(actorStateCts[actorStateCts['actor_country'] == 'Undetermined'].index)

actorStateCtsDrop

fig = px.bar(actorStateCtsDrop,
             y='actor_country',
             x='counts',
             orientation='h',
             color_discrete_sequence=px.colors.qualitative.Dark2,
             labels={'actor_country': 'Actor Country', 'counts': 'Number of Attacks'},
             title='Distribution of Cyber Attacks by Actor Country')

fig.update_layout(
    yaxis_title='Actor Country',
    xaxis_title='Number of Attacks',
    showlegend=False,
    plot_bgcolor='white'
)

fig.show()

actor_country_year = df.groupby(['actor_country',
                                df['event_date'].dt.year]).size().reset_index(name='cyber_events_count')
actor_country_year.rename(columns={'event_date': 'year'}, inplace=True)
actorCountryYr = actor_country_year
actorCountryYr

# Merge actorCountryYr with giniDf based on country and year
actorCountryYr = pd.merge(actorCountryYr, giniDf, left_on=['actor_country', 'year'], right_on=['Country',
'Year'], how='left')

# Rename and drop columns as needed
actorCountryYr.rename(columns={'Gini coefficient': 'Gini Coefficient'}, inplace=True)

```

```

# Create a copy to avoid SettingWithCopyWarning
actorCountryYr2 = actorCountryYr.copy()

# Function to find the closest year in giniDf
def find_closest_year(row):
    country = row['actor_country']
    year = row['year']
    country_data = giniDf[giniDf['Country'] == country]
    if not country_data.empty:
        available_years = country_data['Year'].values
        closest_year = max([y for y in available_years if y <= year], default=np.nan)
        if not pd.isna(closest_year): # Check if closest year was found
            gini_value = country_data[country_data['Year'] == closest_year]['Gini coefficient'].iloc[0]
            return gini_value

    return np.nan # Return NaN if no matching year is found

# Apply the function to find the Gini coefficient
actorCountryYr2['Gini Coefficient'] = actorCountryYr.apply(find_closest_year, axis=1)

# Print the updated dataframe
actorCountryYr2

# Find unique values in actorCountryYr2['actor_country']
actor_countries = actorCountryYr2['actor_country'].unique()

# Find unique values in giniDf['Country']
gini_countries = giniDf['Country'].unique()

# Find values in actor_countries that are not in gini_countries
missing_countries = set(actor_countries) - set(gini_countries)

# Print the missing countries
missing_countries

actorCountryYr3 = actorCountryYr2.replace(['Bolivia (Plurinational State of)'], 'Bolivia')
actorCountryYr3 = actorCountryYr3.replace(['Iran (Islamic Republic of)'], 'Islamic Republic of Iran')
actorCountryYr3 = actorCountryYr3.replace(['Korea (the Republic of)'], 'South Korea')

# Find unique values in actorCountryYr2['actor_country']
actor_countries = actorCountryYr2['actor_country'].unique()

# Find unique values in giniDf['Country']
gini_countries = giniDf2['country_name'].unique()

# Find values in actor_countries that are not in gini_countries
missing_countries = set(actor_countries) - set(gini_countries)

# Print the missing countries
missing_countries

```



```

actorCountryYr2['Gini Coefficient'].notnull().sum()

# Drop rows with NaN values in 'Gini Coefficient' column
actorCountryYr3 = actorCountryYr2.dropna(subset=['Gini Coefficient'])
actorCountryYr3.count()

actorCountryYr3

fig = px.scatter(actorCountryYr3, x="Gini Coefficient", y="cyber_events_count",
                 color="actor_country", hover_data=['year'],
                 title="Cyber Event Count vs. Gini Coefficient by Actor Country and Year")
fig.update_layout(font=dict(size=14))
fig.show()

```

```

actorCountryYr3

# use gini coeficient

# use unemployment

# use gdp

# time series mdoel

```

```

"""# Last Do"""

```

create three pie charts of disruptive vs. exploitive attacks for:

1. Criminal actors in public administration
2. Criminal actors in healthcare and social assistance
3. Criminal actors in information.

This will explain when criminals disrupt vs. exploit. Suspect criminals disrupt in public administration, and then hold the industry to ransom

Graph w/ and w/o protest

```

final_merged_df

```

```

import matplotlib.pyplot as plt
import pandas as pd

```

```

dfActorType[['Criminal', 'Hactivist', 'Hobbyist', 'Nation-State', 'Terrorist', 'Undetermined']].plot(kind='bar',
stacked=True, figsize=(12, 6))
plt.title('Distribution of Actor Types Across Industries')
plt.xlabel('Industry')
plt.ylabel('Number of Cyber Attacks')
plt.xticks(range(len(dfActorType)), dfActorType['Sector Title'], rotation=90)
_ = plt.legend(title='Actor Type')

```

```
#plt.show()
```

```
import matplotlib.pyplot as plt
import pandas as pd
```

```
dfActorType[['Criminal', 'Hobbyist', 'Undetermined']].plot(kind='bar', stacked=True, figsize=(12, 6))
plt.title('Distribution of Actor Types Across Industries without Political-based: Hacktivist, Nation-State,
Terrorist')
plt.xlabel('Industry')
plt.ylabel('Number of Cyber Attacks')
plt.xticks(range(len(dfActorType)), dfActorType['Sector Title'], rotation=90)
_ = plt.legend(title='Actor Type')
#plt.show()
```

```
dfMotive[['Financial' , 'Industrial-Espionage', 'Personal Attack' , 'Political-Espionage' , 'Protest'
, 'Protest, Financial' , 'Protest; Political-Espionage' , 'Sabotage' , 'Undetermined
Motive']].plot(kind='bar', stacked=True, figsize=(12, 6))
plt.title('Distribution of Motive Across All Industries')
plt.xlabel('Industry')
plt.ylabel('Number of Cyber Attacks')
plt.xticks(range(len(dfActorType)), dfActorType['Sector Title'], rotation=90)
_ = plt.legend(title='Motive')
```

```
dfMotive[['Financial' , 'Industrial-Espionage', 'Personal Attack', 'Sabotage' , 'Undetermined
Motive']].plot(kind='bar', stacked=True, figsize=(12, 6))
plt.title('Distribution of Motive Types Across All Industries without Political Based Motives: Political-
Espionage, Protest, Protest, Financial, Protest; Political-Espionage')
plt.xlabel('Industry')
plt.ylabel('Number of Cyber Attacks')
plt.xticks(range(len(dfActorType)), dfActorType['Sector Title'], rotation=90)
_ = plt.legend(title='Actor Type')
```

```
dfMotive[['Financial' , 'Industrial-Espionage', 'Personal Attack', 'Sabotage']].plot(kind='bar', stacked=True,
figsize=(12, 6))
plt.title('Distribution of Determined Motive Types Across All Industries without Political Based Motives:
Political-Espionage, Protest, Protest, Financial, Protest; Political-Espionage')
plt.xlabel('Industry')
plt.ylabel('Number of Cyber Attacks')
plt.xticks(range(len(dfActorType)), dfActorType['Sector Title'], rotation=90)
_ = plt.legend(title='Actor Type')
```

```
#1. Criminal actors in public administration
```

```
public_admin_criminal = dfActorType[dfActorType['Sector Title'] == 'Public Administration']
```

```
public_admin_criminal
```

```
import pandas as pd
import plotly.express as px
```

```

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Criminal', 'Hactivist', 'Hacktivist', 'Hobbyist', 'Nation-State', 'Terrorist', 'Undetermined'],
    'Count': [public_admin_criminal['Criminal'].iloc[0],
              public_admin_criminal['Hactivist'].iloc[0],
              public_admin_criminal['Hacktivist'].iloc[0],
              public_admin_criminal['Hobbyist'].iloc[0],
              public_admin_criminal['Nation-State'].iloc[0],
              public_admin_criminal['Terrorist'].iloc[0],
              public_admin_criminal['Undetermined'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Threat Actors in Public Administration')
fig.show()

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Criminal', 'Hactivist', 'Hacktivist', 'Hobbyist', 'Nation-State', 'Terrorist'],
    'Count': [public_admin_criminal['Criminal'].iloc[0],
              public_admin_criminal['Hactivist'].iloc[0],
              public_admin_criminal['Hacktivist'].iloc[0],
              public_admin_criminal['Hobbyist'].iloc[0],
              public_admin_criminal['Nation-State'].iloc[0],
              public_admin_criminal['Terrorist'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Determined Threat Actors in Public Administration')
fig.show()

# 2. Criminal actors in healthcare and social assistance

public_admin_motive = dfMotive[dfActorType['Sector Title'] == 'Public Administration']

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Financial',
                    'Industrial-Espionage',
                    'Personal Attack',
                    'Political-Espionage',
                    'Protest',

```

```

'Protest,Financial',
'Protest;Political-Espionage',
'Sabotage',
'Undetermined Motive'],
    'Count': [public_admin_motive['Financial'].iloc[0],
               public_admin_motive['Industrial-Espionage'].iloc[0],
               public_admin_motive['Personal Attack'].iloc[0],
               public_admin_motive['Political-Espionage'].iloc[0],
               public_admin_motive['Protest'].iloc[0],
               public_admin_motive['Protest,Financial'].iloc[0],
               public_admin_motive['Protest;Political-Espionage'].iloc[0],
               public_admin_motive['Sabotage'].iloc[0],
               public_admin_motive['Undetermined Motive'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Motives of Attacks on Public Administration')
fig.show()

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Financial',
                    'Industrial-Espionage',
                    'Personal Attack',
                    'Political-Espionage',
                    'Protest',
                    'Protest,Financial',
                    'Protest;Political-Espionage',
                    'Sabotage'],
    'Count': [public_admin_motive['Financial'].iloc[0],
               public_admin_motive['Industrial-Espionage'].iloc[0],
               public_admin_motive['Personal Attack'].iloc[0],
               public_admin_motive['Political-Espionage'].iloc[0],
               public_admin_motive['Protest'].iloc[0],
               public_admin_motive['Protest,Financial'].iloc[0],
               public_admin_motive['Protest;Political-Espionage'].iloc[0],
               public_admin_motive['Sabotage'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Determined Motives of Attacks on Public Administration')
fig.show()

```

```

healthcareCriminal = dfActorType[dfActorType['Sector Title'] == 'Health Care and Social Assistance']

```

```

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Criminal', 'Hactivist', 'Hacktivist', 'Hobbyist', 'Nation-State', 'Terrorist', 'Undetermined'],
    'Count': [healthcareCriminal['Criminal'].iloc[0],
              healthcareCriminal['Hactivist'].iloc[0],
              healthcareCriminal['Hacktivist'].iloc[0],
              healthcareCriminal['Hobbyist'].iloc[0],
              healthcareCriminal['Nation-State'].iloc[0],
              healthcareCriminal['Terrorist'].iloc[0],
              healthcareCriminal['Undetermined'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Threat Actors in Health Care and Social Assistance')
fig.show()

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Criminal', 'Hactivist', 'Hacktivist', 'Hobbyist', 'Nation-State', 'Terrorist'],
    'Count': [healthcareCriminal['Criminal'].iloc[0],
              healthcareCriminal['Hactivist'].iloc[0],
              healthcareCriminal['Hacktivist'].iloc[0],
              healthcareCriminal['Hobbyist'].iloc[0],
              healthcareCriminal['Nation-State'].iloc[0],
              healthcareCriminal['Terrorist'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Determined Threat Actors in Health Care and Social Assistance')
fig.show()

healthcareMotive = dfMotive[dfActorType['Sector Title'] == 'Health Care and Social Assistance']

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Financial',
                    'Industrial-Espionage',
                    'Personal Attack',

```

```

'Political-Espionage',
'Protest',
'Protest,Financial',
'Protest;Political-Espionage',
'Sabotage',
'Undetermined Motive'],
    'Count': [healthcareMotive['Financial'].iloc[0],
               healthcareMotive['Industrial-Espionage'].iloc[0],
               healthcareMotive['Personal Attack'].iloc[0],
               healthcareMotive['Political-Espionage'].iloc[0],
               healthcareMotive['Protest'].iloc[0],
               healthcareMotive['Protest,Financial'].iloc[0],
               healthcareMotive['Protest;Political-Espionage'].iloc[0],
               healthcareMotive['Sabotage'].iloc[0],
               healthcareMotive['Undetermined Motive'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Motives of Attacks on Health Care and Social Assistance')
fig.show()

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Financial',
                    'Industrial-Espionage',
                    'Personal Attack',
                    'Political-Espionage',
                    'Protest',
                    'Protest,Financial',
                    'Protest;Political-Espionage',
                    'Sabotage'],
    'Count': [healthcareMotive['Financial'].iloc[0],
               healthcareMotive['Industrial-Espionage'].iloc[0],
               healthcareMotive['Personal Attack'].iloc[0],
               healthcareMotive['Political-Espionage'].iloc[0],
               healthcareMotive['Protest'].iloc[0],
               healthcareMotive['Protest,Financial'].iloc[0],
               healthcareMotive['Protest;Political-Espionage'].iloc[0],
               healthcareMotive['Sabotage'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Determined Motives of Attacks on Health Care and Social Assistance')
fig.show()

```

```

infoCriminal = dfActorType[dfActorType['Sector Title'] == 'Information']

infoCriminal

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Criminal', 'Hacktivist', 'Hacktivist', 'Hobbyist', 'Nation-State', 'Terrorist', 'Undetermined'],
    'Count': [infoCriminal['Criminal'].iloc[0],
              infoCriminal['Hacktivist'].iloc[0],
              infoCriminal['Hacktivist'].iloc[0],
              infoCriminal['Hobbyist'].iloc[0],
              infoCriminal['Nation-State'].iloc[0],
              infoCriminal['Terrorist'].iloc[0],
              infoCriminal['Undetermined'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Threat Actors in Information')
fig.show()

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Criminal', 'Hacktivist', 'Hacktivist', 'Hobbyist', 'Nation-State', 'Terrorist'],
    'Count': [infoCriminal['Criminal'].iloc[0],
              infoCriminal['Hacktivist'].iloc[0],
              infoCriminal['Hacktivist'].iloc[0],
              infoCriminal['Hobbyist'].iloc[0],
              infoCriminal['Nation-State'].iloc[0],
              infoCriminal['Terrorist'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Determined Threat Actors in Information')
fig.show()

infoMotive = dfMotive[dfActorType['Sector Title'] == 'Information']

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {

```

```

    'Threat Actor': ['Financial',
'Industrial-Espionage',
'Personal Attack',
'Political-Espionage',
'Protest',
'Protest,Financial',
'Protest;Political-Espionage',
'Sabotage',
'Undetermined Motive'],
    'Count': [infoMotive['Financial'].iloc[0],
infoMotive['Industrial-Espionage'].iloc[0],
infoMotive['Personal Attack'].iloc[0],
infoMotive['Political-Espionage'].iloc[0],
infoMotive['Protest'].iloc[0],
infoMotive['Protest,Financial'].iloc[0],
infoMotive['Protest;Political-Espionage'].iloc[0],
infoMotive['Sabotage'].iloc[0],
infoMotive['Undetermined Motive'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Motives of Attacks on Information')
fig.show()

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Financial',
'Industrial-Espionage',
'Personal Attack',
'Political-Espionage',
'Protest',
'Protest,Financial',
'Protest;Political-Espionage',
'Sabotage'],
    'Count': [infoMotive['Financial'].iloc[0],
infoMotive['Industrial-Espionage'].iloc[0],
infoMotive['Personal Attack'].iloc[0],
infoMotive['Political-Espionage'].iloc[0],
infoMotive['Protest'].iloc[0],
infoMotive['Protest,Financial'].iloc[0],
infoMotive['Protest;Political-Espionage'].iloc[0],
infoMotive['Sabotage'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart

```



```

fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Determined Motives of Attacks on
Information')
fig.show()

### exploitative disruptive

typeMotiveDf = df.groupby('event_type')['motive'].value_counts().unstack(fill_value=0).reset_index()

typeActorDf = df.groupby('event_type')['actor_type'].value_counts().unstack(fill_value=0).reset_index()

typeActorDf

disruptiveMotiveDf = typeMotiveDf[typeMotiveDf['event_type'] == 'Disruptive']

disruptiveMotiveDf

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
    'Threat Actor': ['Financial',
                    'Industrial-Espionage',
                    'Personal Attack',
                    'Political-Espionage',
                    'Protest',
                    'Protest,Financial',
                    'Protest;Political-Espionage',
                    'Sabotage'],
    'Count': [disruptiveMotiveDf['Financial'].iloc[0],
              disruptiveMotiveDf['Industrial-Espionage'].iloc[0],
              disruptiveMotiveDf['Personal Attack'].iloc[0],
              disruptiveMotiveDf['Political-Espionage'].iloc[0],
              disruptiveMotiveDf['Protest'].iloc[0],
              disruptiveMotiveDf['Protest,Financial'].iloc[0],
              disruptiveMotiveDf['Protest;Political-Espionage'].iloc[0],
              disruptiveMotiveDf['Sabotage'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Determined Motives of Disruptive Attacks')
fig.show()

exploitiveMotiveDf = typeMotiveDf[typeMotiveDf['event_type'] == 'Exploitive']

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {

```

```

'Threat Actor': ['Financial',
'Industrial-Espionage',
'Personal Attack',
'Political-Espionage',
'Protest',
'Protest,Financial',
'Protest;Political-Espionage',
'Sabotage'],
'Count': [exploitiveMotiveDf['Financial'].iloc[0],
          exploitiveMotiveDf['Industrial-Espionage'].iloc[0],
          exploitiveMotiveDf['Personal Attack'].iloc[0],
          exploitiveMotiveDf['Political-Espionage'].iloc[0],
          exploitiveMotiveDf['Protest'].iloc[0],
          exploitiveMotiveDf['Protest,Financial'].iloc[0],
          exploitiveMotiveDf['Protest;Political-Espionage'].iloc[0],
          exploitiveMotiveDf['Sabotage'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Determined Motives of Exploitive Attacks')
fig.show()

exploitiveActorDf = typeActorDf[typeActorDf['event_type'] == 'Exploitive']

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart
data = {
'Threat Actor': ['Criminal', 'Hacktivist', 'Hacktivist', 'Hobbyist', 'Nation-State', 'Terrorist'],
'Count': [exploitiveActorDf['Criminal'].iloc[0],
          exploitiveActorDf['Hacktivist'].iloc[0],
          exploitiveActorDf['Hacktivist'].iloc[0],
          exploitiveActorDf['Hobbyist'].iloc[0],
          exploitiveActorDf['Nation-State'].iloc[0],
          exploitiveActorDf['Terrorist'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Determined Threat Actors of Exploitive Attacks')
fig.show()

disruptiveActorDf = typeActorDf[typeActorDf['event_type'] == 'Disruptive']

import pandas as pd
import plotly.express as px

# Create a DataFrame for the pie chart

```

```

data = {
    'Threat Actor': ['Criminal', 'Hactivist', 'Hacktivist', 'Hobbyist', 'Nation-State', 'Terrorist'],
    'Count': [disruptiveActorDf['Criminal'].iloc[0],
              disruptiveActorDf['Hactivist'].iloc[0],
              disruptiveActorDf['Hacktivist'].iloc[0],
              disruptiveActorDf['Hobbyist'].iloc[0],
              disruptiveActorDf['Nation-State'].iloc[0],
              disruptiveActorDf['Terrorist'].iloc[0]]
}
df_pie = pd.DataFrame(data)

# Create the pie chart
fig = px.pie(df_pie, values='Count', names='Threat Actor', title='Determined Threat Actors of Disruptive
Attacks')
fig.show()

```

disruptiveActorDf

""""# Country""""

```

actorStateCts = df.groupby('actor_country').size().reset_index(name='counts')
actorStateCts.sort_values(by='counts', ascending=False, inplace=True)
actorStateCts

```

```

affectedCts = df.groupby('country').size().reset_index(name='counts')
affectedCts.sort_values(by='counts', ascending=False, inplace=True)
affectedCts

```

```

## poverty and inequality indicators
# pip.csv from https://pip.worldbank.org/poverty-calculator

```

```

# Commented out IPython magic to ensure Python compatibility.
drive.mount('/content/drive')
# %cd /content/drive/My Drive/GoogColab/effectsBreach/data
pipDf = pd.read_csv('pip.csv', encoding='latin-1')

```

```

import matplotlib.pyplot as plt

```

```

top_10_countries = affectedCts.head(10)

```

```

plt.figure(figsize=(12, 6))
bars = plt.bar(top_10_countries['country'], top_10_countries['counts'])
plt.xlabel('Country')
plt.ylabel('Counts')
plt.title('Top 10 Affected Countries')
plt.xticks(rotation=45, ha='right')

```

```

for bar in bars:
    yval = bar.get_height()

```

```

plt.text(bar.get_x() + bar.get_width()/2, yval, int(yval), va='bottom', ha='center')

plt.tight_layout()
plt.show()

import matplotlib.pyplot as plt

actorStateCts_filtered = actorStateCts[actorStateCts['actor_country'] != 'undefined']

top_10_countries = actorStateCts_filtered.sort_values(by='counts', ascending=False).head(10)

plt.figure(figsize=(12, 6))
bars = plt.bar(top_10_countries['actor_country'], top_10_countries['counts'])
plt.xlabel('Country')
plt.ylabel('Counts')
plt.title('Top 10 Actor Countries (excluding "undefined")')
plt.xticks(rotation=45, ha='right')

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval, int(yval), va='bottom', ha='center')

plt.tight_layout()
plt.show()

pipDf.info()

pipDf

```

"""# Potential Regression Analyses

Predicting Event Frequency over time:

we can use year and month as predictors to do the model. Linear regression. How has the number of cybersecurity incidents changed over time?

Impact of Actor Type on event subtypes. We will use actor_type as a categorical variable to predict event subtype. Logistic Regression. Are certain actor types more likely to perform specific types of events

Using country and motive to predict event frequency or the likelihood of a specific event type. Linear Regression. Does the country or motive significantly influence the number of incidents?

"""

```

import matplotlib as plt
import seaborn as sns

df = pd.read_csv('/content/mainData.csv', encoding='latin-1')

df.head()

```

```

df['event_date'] = pd.to_datetime(df['event_date'], format = 'mixed')

df.head()

event_frequency = df.groupby(df['event_date'].dt.to_period('M')).size().reset_index(name='event_count')

event_frequency['event_date'] = event_frequency['event_date'].dt.to_timestamp()

event_frequency.head()

import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(12,6))
sns.lineplot(data = event_frequency,x='event_date',y='event_count',marker='o')
plt.title('Event Frequency Over Time')
plt.xlabel('Date')
plt.ylabel('Event Count')
plt.grid(True)
plt.show()

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,r2_score
import numpy as np

event_frequency['timestamp'] = event_frequency['event_date'].map(lambda x:x.toordinal())
X = event_frequency[['timestamp']]
y = event_frequency['event_count']

X

y

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

model = LinearRegression()
model.fit(X_train,y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test,y_pred)
r2 = r2_score(y_test,y_pred)

model_coefficients = model.coef_
model_intercept = model.intercept_

mse

r2

model_coefficients

```

```

model_intercept

plt.figure(figsize=(12,6))
plt.scatter(event_frequency['timestamp'],event_frequency['event_count'],color='red',alpha=0.5,label='Actual
Data')
regression_line = model.predict(event_frequency[['timestamp']])
plt.plot(event_frequency['timestamp'],regression_line,color='green',linewidth=2,label='Regression Line')

df['actor_type'].unique()

df['actor_type'] = df['actor_type'].replace('Hacktivist','Hactivist')
df['actor_type'].unique()

df_encoded = pd.get_dummies(df,columns=['actor_type','event_type'],drop_first=True)

df_encoded

df_encoded.columns

df_encoded.columns = df_encoded.columns.str.strip()

grouped_data = df_encoded.groupby(df_encoded['event_date'].dt.to_period('M')).agg({
    'event_date': 'first',
    'actor_type_Nation-State': 'sum',
    'actor_type_Hacktivist': 'sum',
    'actor_type_Terrorist': 'sum',
    'actor_type_Undetermined': 'sum',
    'actor_type_Hobbyist': 'sum',
    'event_type_Disruptive': 'sum',
    'event_type_Exploitive': 'sum',
    'event_type_Mixed': 'sum',
    'event_type_Undetermined': 'sum',
    'slug': 'count'
}).rename(columns={'slug': 'event_count'})

grouped_data.head()

grouped_data.index = grouped_data.index.to_timestamp()
grouped_data['timestamp'] = grouped_data.index.map(lambda x:x.toordinal())

grouped_data.columns

X = grouped_data[['timestamp','actor_type_Nation-State','actor_type_Hacktivist','actor_type_Terrorist',
'actor_type_Undetermined',
    'actor_type_Hobbyist','event_type_Disruptive','event_type_Exploitive',
    'event_type_Mixed','event_type_Undetermined']]
y = grouped_data['event_count']

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

```

```

model = LinearRegression()
model.fit(X_train,y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test,y_pred)
r2 = r2_score(y_test,y_pred)

mse

r2

model.coef_

model.intercept_

"""## Country and Motive Regression"""

df_encoded = pd.get_dummies(df,columns=['country','motive'],drop_first=True)

df_encoded.columns

grouped_data = df_encoded.groupby(df_encoded['event_date'].dt.to_period('M')).agg({
    'event_date': 'first',
    'slug': 'count',
    **{col: 'sum' for col in df_encoded.columns if col.startswith('country_')},
    **{col: 'sum' for col in df_encoded.columns if col.startswith('motive_')}
}).rename(columns={'slug': 'event_count'})

grouped_data.index = grouped_data.index.to_timestamp()

X = grouped_data.drop(columns=['event_date','event_count'])
y = grouped_data['event_count']

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)

model = LinearRegression()
model.fit(X_train,y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test,y_pred)
r2 = r2_score(y_test,y_pred)

mse

r2

model.coef_

model.intercept_

```

```
"""## country and motive prediction"""
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.multioutput import MultiOutputRegressor
```

```
df_encoded = pd.get_dummies(df,columns=['country','motive'],drop_first=True)
```

```
X = df_encoded.filter(regex='^country_')
y = df_encoded.filter(regex='^motive_')
```

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

```
logistic_model = MultiOutputRegressor(LogisticRegression(max_iter=1000))
logistic_model.fit(X_train,y_train)
```

```
y_pred = logistic_model.predict(X_test)
```

```
accuracy_scores = (y_pred == y_test.values).mean(axis=0)
```

```
accuracy_scores
```

```
logistic_model.estimators_
```

```
df['motive'].unique()
```

```
pip_data = pd.read_csv("pip.csv")
pip_data.head()
```

```
pip_data['country_name'] = pip_data['country_name'].str.strip()
df['country'] = df['country'].str.strip()
```

```
attacks_per_country = df.groupby('country').size().reset_index(name='num_attacks')
```

```
attacks_per_country
```

```
gdp_data = pip_data[['country_name','reporting_gdp']].groupby('country_name').mean().reset_index()
```

```
gdp_data
```

```
merged_data = attacks_per_country.merge(gdp_data,left_on='country',right_on='country_name',how='inner')
```

```
merged_data
```

```
merged_data = merged_data.drop(columns=['country_name'])
```

```
merged_data
```

```
from scipy.stats import pearsonr
```

```
correlation, p_value = pearsonr(merged_data['num_attacks'],merged_data['reporting_gdp'])
```



```

correlation_res = {"Correlation Coefficient":correlation,"p value":p_value}

correlation_res

import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plt.scatter(merged_data['reporting_gdp'],merged_data['num_attacks'],alpha=0.7)
plt.xlabel('GDP')
plt.ylabel('Number of attacks')
plt.grid(True,linestyle='--',alpha=0.6)
plt.show()

import statsmodels.api as sm

X = merged_data['reporting_gdp']
y = merged_data['num_attacks']

X_with_const = sm.add_constant(X)

model = sm.OLS(y,X_with_const).fit()

regression_summary = model.summary()
regression_summary

from scipy.stats import ttest_ind

median_gdp = merged_data['reporting_gdp'].median()
high_gdp = merged_data[merged_data['reporting_gdp'] > median_gdp]['num_attacks']
low_gdp = merged_data[merged_data['reporting_gdp'] <= median_gdp]['num_attacks']

t_stat,p_value_ttest = ttest_ind(high_gdp,low_gdp,equal_var=False)

t_stat

p_value_ttest

pip_data.columns

pipData = pip_data[['country_name','reporting_gdp', 'poverty_gap', 'cpi','gini', 'polarization'
]].groupby('country_name').mean().reset_index()

merged2 = attacks_per_country.merge(pipData,left_on='country',right_on='country_name',how='inner')

merged2 = merged2.drop(columns=['country_name'])

merged2

plt.figure(figsize=(12, 10))
independent_vars = ['reporting_gdp', 'poverty_gap', 'cpi', 'gini', 'polarization']
for i, var in enumerate(independent_vars, 1):

```

```

plt.subplot(2, 3, i)
sns.scatterplot(data=merged2, x=var, y='num_attacks')
plt.title(f'num_attacks vs {var}')
plt.xlabel(var)
plt.ylabel('num_attacks')
plt.tight_layout()
plt.show()

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.optimize import curve_fit

def exp_func(x, a, b):
    return a * np.exp(b * x)

plt.figure(figsize=(12, 10))

independent_vars = ['reporting_gdp', 'poverty_gap', 'cpi', 'gini', 'polarization']

for i, var in enumerate(independent_vars, 1):
    plt.subplot(2, 3, i)
    sns.scatterplot(data=merged2, x=var, y='num_attacks')

    popt, _ = curve_fit(exp_func, merged2[var], merged2['num_attacks'], maxfev=10000)
    a, b = popt

    x_vals = np.linspace(merged2[var].min(), merged2[var].max(), 100)
    y_vals = exp_func(x_vals, *popt)

    plt.plot(x_vals, y_vals, color='red', label=f'Exp Fit: $y = {a:.2f} e^{{{{b:.2f}}x}}$')

    predicted_y = exp_func(merged2[var], a, b)
    ss_res = np.sum((merged2['num_attacks'] - predicted_y) ** 2)
    ss_tot = np.sum((merged2['num_attacks'] - np.mean(merged2['num_attacks'])) ** 2)
    r2 = 1 - (ss_res / ss_tot)

    plt.title(f'num_attacks vs {var}\nExp Fit: a={a:.2f}, b={b:.2f}, R^2={r2:.2f}')
    plt.xlabel(var)
    plt.ylabel('num_attacks')
    plt.legend()

plt.tight_layout()
plt.show()

gdp_bins = pd.qcut(merged_data['reporting_gdp'], q=3, labels=['Low GDP', 'Medium GDP', 'High GDP'])

merged_data['GDP_Range'] = gdp_bins

```

```

gdp_range_comparison =
merged_data.groupby('GDP_Range')['num_attacks'].agg(['mean','count','std']).reset_index()

gdp_range_comparison

plt.figure(figsize=(8,6))
plt.title("Average Number of Attacks on Country by GDP Range")
sns.barplot(data=gdp_range_comparison,x='GDP_Range',y='mean')
plt.xlabel('GDP Range')
plt.ylabel('Avg Num Attacks')

actor_country_attacks = df.groupby('actor_country').size().reset_index(name='num_attacks')

gdp_data = pip_data[['country_name','reporting_gdp']].groupby('country_name').mean().reset_index()

merged_actor_data =
actor_country_attacks.merge(gdp_data,left_on='actor_country',right_on='country_name',how='inner')

merged_actor_data

merged_actor_data = merged_actor_data.drop(columns=['country_name'])

merged_actor_data

correlation,p_value = pearsonr(merged_actor_data['num_attacks'],merged_actor_data['reporting_gdp'])

correlation

p_value

pip_data.columns

attacks_per_country = df.groupby('country').size().reset_index(name='num_attacks')

gini_data = pip_data[['country_name','gini']].groupby('country_name').mean().reset_index()

merged_centry_data =
attacks_per_country.merge(gini_data,left_on='country',right_on='country_name',how='inner')

merged_centry_data

merged_centry_data = merged_centry_data.drop(columns=['country_name'])
merged_centry_data

correlation,p_value = pearsonr(merged_centry_data['num_attacks'],merged_centry_data['gini'])

correlation

p_value

median_gini = merged_centry_data['gini'].median()

```

```

high_gini = merged_cntry_data[merged_cntry_data['gini'] > median_gini]['num_attacks']
low_gini = merged_cntry_data[merged_cntry_data['gini'] <= median_gini]['num_attacks']

t_stat,p_value_ttest = ttest_ind(high_gini,low_gini,equal_var = False)

t_stat

p_value_ttest

gini_bins = pd.qcut(merged_cntry_data['gini'],q=3,labels=['Low Gini','Medium Gini','High Gini'])

merged_cntry_data['Gini_Range'] = gini_bins

gini_range_comparison =
merged_cntry_data.groupby('Gini_Range')['num_attacks'].agg(['mean','count','std']).reset_index()

gini_range_comparison

plt.figure(figsize=(8,6))
plt.title("Average Number of Attacks on Country by Gini Range")
sns.barplot(data=gini_range_comparison,x='Gini_Range',y='mean')
plt.xlabel('Gini Range')
plt.ylabel('Avg Number of Attacks')

merged_actor_cntry_data =
actor_country_attacks.merge(gini_data,left_on='actor_country',right_on='country_name',how='inner')

merged_actor_cntry_data

merged_actor_cntry_data = merged_actor_cntry_data.drop(columns=['country_name'])

merged_actor_cntry_data

correlation,p_value = pearsonr(merged_actor_cntry_data['num_attacks'],merged_actor_cntry_data['gini'])

correlation

p_value

median_gini = merged_actor_cntry_data['gini'].median()
high_gini = merged_actor_cntry_data[merged_actor_cntry_data['gini'] > median_gini]['num_attacks']
low_gini = merged_actor_cntry_data[merged_actor_cntry_data['gini'] <= median_gini]['num_attacks']

t_stat,p_value_ttest = ttest_ind(high_gini,low_gini,equal_var = False)

t_stat

p_value_ttest

```

```

"""# Time Series"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# Commented out IPython magic to ensure Python compatibility.
from google.colab import drive
drive.mount('/content/drive')
# %cd /content/drive/My Drive/GoogColab/effectsBreach/data

df = pd.read_csv('mainData.csv', encoding='latin-1')
df['event_date'] = pd.to_datetime(df['event_date'], format='mixed')

df.set_index('event_date', inplace=True)
freq5Df = df.resample('5D').size().reset_index(name='count')
freq10Df = df.resample('10D').size().reset_index(name='count')
freq30Df = df.resample('30D').size().reset_index(name='count')
freq90Df = df.resample('90D').size().reset_index(name='count')

freq10Df.drop(freq10Df.tail(1).index, inplace=True)
freq5Df.drop(freq5Df.tail(1).index, inplace=True)
freq30Df.drop(freq30Df.tail(1).index, inplace=True)
freq90Df.drop(freq90Df.tail(1).index, inplace=True)

freq5Df

pip install pmdarima

# Predictive Modeling
from pmdarima import auto_arima

data = freq5Df
data['event_date'] = pd.to_datetime(data['event_date'])
data = data.set_index('event_date')
data = data.resample('D').mean().interpolate()

train = data[:'2024-06-01']
test = data['2024-06-02:']

model = auto_arima(train['count'], seasonal=False, trace=True)

forecast = model.predict(n_periods=len(test))

# Predictive Modeling Visualisation
plt.plot(train.index, train['count'], label='Train')
plt.plot(test.index, test['count'], label='Test')
plt.plot(test.index, forecast, label='Forecast', color='red')
plt.legend()
plt.show()

```

```
# zoom in
train_filtered = train[train.index >= '2024']
test_filtered = test[test.index >= '2024']
forecast_filtered = forecast[test.index >= '2024']

plt.plot(train_filtered.index, train_filtered['count'], label='Train')
plt.plot(test_filtered.index, test_filtered['count'], label='Test')
plt.plot(test_filtered.index, forecast_filtered, label='Forecast', color='red')
plt.legend()
plt.show()
```

```
# 10d
# Predictive Modeling
from pmdarima import auto_arima
```

```
data = freq10Df
data['event_date'] = pd.to_datetime(data['event_date'])
data = data.set_index('event_date')
data = data.resample('D').mean().interpolate()
```

```
train = data[:'2024-06-01']
test = data['2024-06-02:']
```

```
model = auto_arima(train['count'],seasonal=False,trace=True)
```

```
forecast = model.predict(n_periods=len(test))
```

```
# Predictive Modeling Visualisation
plt.plot(train.index,train['count'],label='Train')
plt.plot(test.index,test['count'],label='Test')
plt.plot(test.index,forecast,label='Forecast',color='red')
plt.legend()
plt.show()
```

```
# zoom in
train_filtered = train[train.index >= '2024']
test_filtered = test[test.index >= '2024']
forecast_filtered = forecast[test.index >= '2024']

plt.plot(train_filtered.index, train_filtered['count'], label='Train')
plt.plot(test_filtered.index, test_filtered['count'], label='Test')
plt.plot(test_filtered.index, forecast_filtered, label='Forecast', color='red')
plt.legend()
plt.show()
```

```
freq30Df
sorted = freq30Df.sort_values(by='count')
sorted
```

```

# 30
# Predictive Modeling
from pmdarima import auto_arima

data = freq30Df
data['event_date'] = pd.to_datetime(data['event_date'])
data = data.set_index('event_date')
data = data.resample('D').mean().interpolate()

train = data[:'2024-06-01']
test = data['2024-06-02:']

model = auto_arima(train['count'],seasonal=False,trace=True)

forecast = model.predict(n_periods=len(test))

# Predictive Modeling Visualisation
plt.plot(train.index,train['count'],label='Train')
plt.plot(test.index,test['count'],label='Test')
plt.plot(test.index,forecast,label='Forecast',color='red')
plt.legend()
plt.show()

from pmdarima import auto_arima
import pandas as pd

data = freq30Df
data['event_date'] = pd.to_datetime(data['event_date'])
data = data.set_index('event_date')
data = data.resample('D').mean().interpolate()

train = data[:'2024-06-01']

model = auto_arima(train['count'], seasonal=False, trace=True)

last_date = train.index[-1]
forecast_end_date = '2025-06-30'
n_days_to_forecast = (pd.to_datetime(forecast_end_date) - last_date).days

forecast = model.predict(n_periods=n_days_to_forecast)

forecast_dates = pd.date_range(start=last_date + pd.Timedelta(days=1), periods=n_days_to_forecast)
forecast_df = pd.DataFrame({'forecast': forecast, index=forecast_dates})

```

```

forecast_combined = pd.concat([data, forecast_df])

print(forecast_combined)

plt.figure(figsize=(12, 6))
plt.plot(data.index, data['count'], label='Observed Data', color='blue')
plt.plot(forecast_df.index, forecast_df['forecast'], label='Forecast', color='orange', linestyle='--')
plt.axvline(x=last_date, color='red', linestyle='--', label='Forecast Start')
plt.title('Forecasting Future Values Until Mid-2025')
plt.xlabel('Date')
plt.ylabel('Count')
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

```

90

```

predictions_df = pd.DataFrame({
    'Date': test.index,
    'Actual': test['count'],
    'Predicted': forecast
}).reset_index(drop=True)

```

predictions_df

```

plt.figure(figsize=(12, 8))
plt.plot(freq5Df["event_date"], freq5Df["count"], label="Frequency 5")
plt.plot(freq10Df["event_date"], freq10Df["count"], label="Frequency 10")
plt.plot(freq30Df["event_date"], freq30Df["count"], label="Frequency 30")
plt.plot(freq90Df["event_date"], freq90Df["count"], label="Frequency 90")
plt.xlabel("Date")
plt.ylabel("Frequency Count")
plt.title("Frequency Count vs Date")
plt.legend()
plt.grid(True)
plt.show()

```

s30.values

```

plt.figure(figsize=(12, 8))
plt.plot(freq5Df["event_date"], freq5Df["count"], label="Frequency 5")
plt.plot(freq10Df["event_date"], freq10Df["count"], label="Frequency 10")
plt.xlabel("Date")
plt.ylabel("Frequency Count")
plt.title("Frequency Count vs Date")
plt.legend()
plt.grid(True)
plt.show()

```



```

from pandas import read_csv
import pandas
from matplotlib import pyplot
from pandas.plotting import autocorrelation_plot

autocorrelation_plot(freq10Df['count'], label = '10 days')
autocorrelation_plot(freq5Df['count'], label = '5 days')
autocorrelation_plot(freq30Df['count'], label = '30 days')
autocorrelation_plot(freq90Df['count'], label = '90 days')
plt.ylim(-0.5, 1.0)
plt.legend()
plt.show()

from statsmodels.tsa.arima.model import ARIMA
from matplotlib import pyplot
## arima on 10day period
s10 = freq10Df.set_index('event_date')
s10['count'] = (s10['count'] - s10['count'].min()) / (s10['count'].max() - s10['count'].min())
model = ARIMA(s10, order=(3,1,4))
model_fit = model.fit()
print(model_fit.summary())

residuals = pd.DataFrame(model_fit.resid)
residuals.plot()
pyplot.show()

residuals.plot(kind='kde')
pyplot.show()

from sklearn.metrics import mean_squared_error
from math import sqrt
X = s10.values
size = int(len(X) * 0.66)
train, test = X[0:size], X[size:len(X)]
history = [x for x in train]
predictions = list()
# walk-forward validation
for t in range(len(test)):
    model = ARIMA(history, order=(3,1,4))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = test[t]
    history.append(obs)
    print('predicted=%f, expected=%f' % (yhat, obs))
# evaluate forecasts
rmse = sqrt(mean_squared_error(test, predictions))
print('Test RMSE: %.3f' % rmse)
# plot forecasts against actual outcomes
pyplot.plot(test)

```

```
pyplot.plot(predictions, color='red')
pyplot.show()
```

```
!pip install pmdarima
```

```
import pandas as pd
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from pmdarima import auto_arima
```

```
auto_model = auto_arima(
    s10,
    seasonal=False,      # Set to True if seasonal component is suspected
    trace=True,          # Prints model details
    error_action='ignore', # Ignores convergence errors
    suppress_warnings=True, # Suppresses warnings
    stepwise=True        # Stepwise search for faster computation
)
print(f'Selected ARIMA order: {auto_model.order}')
print("Fitting the ARIMA model...")
model = ARIMA(s10, order=auto_model.order)
model_fit = model.fit()
print(model_fit.summary())
```

```
X = s10.values
size = int(len(X) * 0.66)
train, test = X[0:size], X[size:len(X)]
history = [x for x in train]
predictions = list()
# walk-forward validation
for t in range(len(test)):
    model = ARIMA(history, order=(0,1,1))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = test[t]
    history.append(obs)
    print('predicted=%f, expected=%f' % (yhat, obs))
# evaluate forecasts
rmse = sqrt(mean_squared_error(test, predictions))
print('Test RMSE: %.3f' % rmse)
# plot forecasts against actual outcomes
pyplot.plot(test, label = 'data')
pyplot.plot(predictions, color='red', label = 'predictions')
pyplot.show()
```

```
# Updated
from statsmodels.tsa.arima.model import ARIMA
from sklearn.preprocessing import StandardScaler
from pmdarima import auto_arima
```

```

from matplotlib import pyplot as plt
from math import sqrt
from sklearn.metrics import mean_squared_error
import numpy as np

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X.reshape(-1,1)).flatten()

size = int(len(X_scaled) * 0.66)
train,test = X_scaled[0:size], X_scaled[size:]

auto_model = auto_arima(train,seasonal=False,trace=True)
order = auto_model.order

history = [x for x in train]
predictions = []

for t in range(len(test)):
    model = ARIMA(history,order=order)
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = test[t]
    history.append(obs)

rmse = sqrt(mean_squared_error(test,predictions))
print('Test RMSE',rmse)

plt.plot(test,label='Data')
plt.plot(predictions,color='red',label='Predictions')
plt.legend()
plt.show()

s10

s30 = freq30Df.set_index('event_date')
s30['count'] = (s30['count'] - s30['count'].min()) / (s30['count'].max() - s30['count'].min())

auto_model = auto_arima(
    s30,
    seasonal=True,          # Set to True if seasonal component is suspected
    trace=True,             # Prints model details
    error_action='ignore',  # Ignores convergence errors
    suppress_warnings=True, # Suppresses warnings
    stepwise=True           # Stepwise search for faster computation
)
print(f'Selected ARIMA order: {auto_model.order}')
print("Fitting the ARIMA model...")
model = ARIMA(s30, order=auto_model.order)

```

```

model_fit = model.fit()
print(model_fit.summary())

X = s30.values
size = int(len(X) * 0.66)
train, test = X[0:size], X[size:len(X)]
history = [x for x in train]
predictions = list()
# walk-forward validation
for t in range(len(test)):
    model = ARIMA(history, order=(0,1,1))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = test[t]
    history.append(obs)
    print('predicted=%f, expected=%f' % (yhat, obs))
# evaluate forecasts
rmse = sqrt(mean_squared_error(test, predictions))
print('Test RMSE: %.3f' % rmse)
# plot forecasts against actual outcomes
pyplot.plot(test, label = 'data')
pyplot.plot(predictions, color='red', label = 'prediction')
pyplot.legend()
pyplot.show()

X = s30.values
size = int(len(X) * 0.66)
train, test = X[0:size], X[size:len(X)]
history = [x for x in train]
predictions = list()
# walk-forward validation
for t in range(len(test)):
    model = ARIMA(history, order=(2,1,1))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = test[t]
    history.append(obs)
    print('predicted=%f, expected=%f' % (yhat, obs))
# evaluate forecasts
rmse = sqrt(mean_squared_error(test, predictions))
print('Test RMSE: %.3f' % rmse)
# plot forecasts against actual outcomes
pyplot.plot(test, label = 'data')
pyplot.plot(predictions, color='red', label = 'prediction')
pyplot.legend()
pyplot.show()

```

```

s90 = freq90Df.set_index('event_date')
s90['count'] = (s90['count'] - s90['count'].min()) / (s90['count'].max() - s90['count'].min())

auto_model = auto_arima(
    s90,
    seasonal=True,          # Set to True if seasonal component is suspected
    trace=True,             # Prints model details
    error_action='ignore',  # Ignores convergence errors
    suppress_warnings=True, # Suppresses warnings
    stepwise=True           # Stepwise search for faster computation
)
print(f'Selected ARIMA order: {auto_model.order}')
print("Fitting the ARIMA model...")
model = ARIMA(s90, order=auto_model.order)
model_fit = model.fit()
print(model_fit.summary())

X = s90.values
size = int(len(X) * 0.66)
train, test = X[0:size], X[size:len(X)]
history = [x for x in train]
predictions = list()
# walk-forward validation
for t in range(len(test)):
    model = ARIMA(history, order=(0,1,0))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = test[t]
    history.append(obs)
    print('predicted=%f, expected=%f' % (yhat, obs))
# evaluate forecasts
rmse = sqrt(mean_squared_error(test, predictions))
print('Test RMSE: %.3f' % rmse)
# plot forecasts against actual outcomes
pyplot.plot(test, label = 'data')
pyplot.plot(predictions, color='red', label = 'prediction')
pyplot.legend()
pyplot.show()

"""# Public Administration"""

df_public_admin = df[df['industry'] == 'Public Administration']

PAfreq10Df = df_public_admin.resample('10D').size().reset_index(name='count')
PAfreq30Df = df_public_admin.resample('30D').size().reset_index(name='count')

plt.plot(PAfreq10Df["event_date"], PAfreq10Df["count"], label="Frequency 10")

```

```

plt.plot(PAfreq30Df["event_date"], PAfreq30Df["count"], label="Frequency 30")
plt.xlabel("Date")
plt.ylabel("Frequency Count")
plt.title("Frequency Count vs Date")
plt.legend()
plt.grid(True)
plt.show()

from pandas import read_csv
import pandas
from matplotlib import pyplot
from pandas.plotting import autocorrelation_plot

autocorrelation_plot(PAfreq10Df['count'], label = '10 days')
autocorrelation_plot(PAfreq30Df['count'], label = '30 days')
plt.legend()
plt.show()

PAs10 = PAfreq10Df.set_index('event_date')
PAs10['count'] = (PAs10['count'] - PAs10['count'].min()) / (PAs10['count'].max() - PAs10['count'].min())

import pandas as pd
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from pmdarima import auto_arima

auto_model = auto_arima(
    PAs10,
    seasonal=True,          # Set to True if seasonal component is suspected
    trace=True,             # Prints model details
    error_action='ignore',  # Ignores convergence errors
    suppress_warnings=True, # Suppresses warnings
    stepwise=True           # Stepwise search for faster computation
)
print(f"Selected ARIMA order: {auto_model.order}")
print("Fitting the ARIMA model...")
model = ARIMA(PAs10, order=auto_model.order)
model_fit = model.fit()
print(model_fit.summary())

import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

residuals = model_fit.resid
plt.figure(figsize=(12, 6))
plt.subplot(211)
plt.plot(residuals)
plt.title("Residuals")
plt.subplot(212)
plt.hist(residuals, bins=20)

```

```
plt.title("Residual Histogram")
plt.tight_layout()
plt.show()
```

```
plot_acf(residuals, lags=30)
plt.title("Residuals ACF")
plt.show()
```

```
plot_pacf(residuals, lags=30)
plt.title("Residuals PACF")
plt.show()
```

```
X = PAs10.values
size = int(len(X) * 0.66)
train, test = X[0:size], X[size:len(X)]
history = [x for x in train]
predictions = list()
# walk-forward validation
for t in range(len(test)):
    model = ARIMA(history, order=(1,1,1))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = test[t]
    history.append(obs)
    print('predicted=%f, expected=%f' % (yhat, obs))
# evaluate forecasts
rmse = sqrt(mean_squared_error(test, predictions))
print('Test RMSE: %.3f' % rmse)
# plot forecasts against actual outcomes
pyplot.plot(test, label = 'data')
pyplot.plot(predictions, color='red', label = 'prediction')
pyplot.legend()
pyplot.show()
```

```
PAs30 = PAsfreq30Df.set_index('event_date')
PAs30['count'] = (PAs30['count'] - PAs30['count'].min()) / (PAs30['count'].max() - PAs30['count'].min())
```

```
import pandas as pd
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from pmdarima import auto_arima
```

```
auto_model = auto_arima(
    PAs30,
    seasonal=True,          # Set to True if seasonal component is suspected
    trace=True,             # Prints model details
    error_action='ignore',  # Ignores convergence errors
    suppress_warnings=True, # Suppresses warnings
```

```

    stepwise=True          # Stepwise search for faster computation
)
print(f'Selected ARIMA order: {auto_model.order}')
print("Fitting the ARIMA model...")
model = ARIMA(PAs30, order=auto_model.order)
model_fit = model.fit()
print(model_fit.summary())

import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

residuals = model_fit.resid
plt.figure(figsize=(12, 6))
plt.subplot(211)
plt.plot(residuals)
plt.title("Residuals")
plt.subplot(212)
plt.hist(residuals, bins=20)
plt.title("Residual Histogram")
plt.tight_layout()
plt.show()

plot_acf(residuals, lags=30)
plt.title("Residuals ACF")
plt.show()

plot_pacf(residuals, lags=30)
plt.title("Residuals PACF")
plt.show()

X = PAs30.values
size = int(len(X) * 0.66)
train, test = X[0:size], X[size:len(X)]
history = [x for x in train]
predictions = list()
# walk-forward validation
for t in range(len(test)):
    model = ARIMA(history, order=(0,1,1))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = test[t]
    history.append(obs)
    print('predicted=%f, expected=%f % (yhat, obs))
# evaluate forecasts
rmse = sqrt(mean_squared_error(test, predictions))
print('Test RMSE: %.3f % rmse)
# plot forecasts against actual outcomes

```



```
pyplot.plot(test, label = 'data')  
pyplot.plot(predictions, color='red', label = 'prediction')  
pyplot.legend()  
pyplot.show()
```